TÉCNICO
UNIVERSIDADE
DE LISBOA

A Process Framework for Human-AI Collaboration in the Software Development Lifecycle

Design and Instantiation of a Supporting Tool

Tomás dos Santos Taborda

Thesis to obtain the Master of Science Degree in

Computer Science and Engineering

Supervisors: Prof. Miguel Mira da Silva
Hugo de Sousa

October 2026

Declaration
I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa. I acknowledge the use of artificial intelligence tools to support the drafting and refinement of this thesis, with all content produced by these tools carefully reviewed, edited, and validated to ensure accuracy and originality.

Acknowledgments

I would like to thank my parents for their friendship, encouragement and caring over all these years, for always being there for me through thick and thin and without whom this project would not be possible. I would also like to thank my grandparents, aunts, uncles and cousins for their understanding and support throughout all these years.

Quisque facilisis erat a dui. Nam malesuada ornare dolor. Cras gravida, diam sit amet rhoncus ornare, erat elit consectetuer erat, id egestas pede nibh eget odio. Proin tincidunt, velit vel porta elementum, magna diam molestie sapien, non aliquet massa pede eu diam. Aliquam iaculis.

Fusce et ipsum et nulla tristique facilisis. Donec eget sem sit amet ligula viverra gravida. Etiam vehicula urna vel turpis. Suspendisse sagittis ante a urna. Morbi a est quis orci consequat rutrum. Nullam egestas feugiat felis. Integer adipiscing semper ligula. Nunc molestie, nisl sit amet cursus convallis, sapien lectus pretium metus, vitae pretium enim wisi id lectus.

Donec vestibulum. Etiam vel nibh. Nulla facilisi. Mauris pharetra. Donec augue. Fusce ultrices, neque id dignissim ultrices, tellus mauris dictum elit, vel lacinia enim metus eu nunc.

I would also like to acknowledge my dissertation supervisors Prof. Some Name and Prof. Some Other Name for their insight, support and sharing of knowledge that has made this Thesis possible.

Last but not least, to all my friends and colleagues that helped me grow as a person and were always there for me during the good and bad times in my life. Thank you.

To each and every one of you - Thank you.

Abstract

Generative Artificial Intelligence and Large Language Models have moved from autocomplete assistants to active contributors across the Software Development Life Cycle, enabling intent-driven development in which practitioners express requirements in natural language and supervise the resulting artefacts. Adoption, however, has outpaced method. A Systematic Literature Review conducted for this work, following Kitchenham's guidelines over 646 retrieved articles, confirms substantial reported benefits alongside recurring risks in reliability, security, traceability and accountability, and identifies the absence of structured, repeatable guidance for integrating these tools into professional workflows as the field's central unresolved problem. Independent evidence sharpens the point: a majority of functionally correct AI-generated code has been reported to carry security defects, and measured delivery bottlenecks shift towards review rather than disappearing, indicating that the value of these tools is conditional on process rather than on model capability alone.

This dissertation addresses that gap through the Design Science Research Methodology. It proposes a process-oriented framework for governed Human-AI collaboration that treats AI systems as first-class collaborators whose contributions must be explicitly bounded, validated and traced. The framework defines lifecycle phases with phase-specific quality gates, responsibilities expressed as functions rather than job titles, and governance metrics, anchored by a persistent, versioned specification that keeps every generated artefact linked to its originating requirement. The framework is instantiated as Apex, a working web application that operationalises each phase, gate and artefact against a real project management backend, and which serves as the demonstration vehicle in a practical setting. Evaluation combines application of the artefact in a real-world project, standardised instruments assessing perceived usability and cognitive workload, semi-structured interviews with practitioners and Agile experts, and an analytical assessment against defined success criteria. The expected contribution is a defensible, evidence-grounded process model that allows organisations to adopt AI assistance without

surrendering oversight of software quality, security and accountability.

Keywords

Human-AI Collaboration; Software Development Life Cycle; Large Language Models; Generative Artificial Intelligence; Process Framework; AI Governance; Design Science Research; Spec-Driven Development; Vibe Coding; Software Engineering

Resumo

A Inteligência Artificial Generativa e os Modelos de Linguagem de Grande Dimensão deixaram de ser meros assistentes de preenchimento automático para se tornarem contribuidores activos ao longo do Ciclo de Vida de Desenvolvimento de Software, permitindo um desenvolvimento orientado à intenção, no qual os profissionais exprimem requisitos em linguagem natural e supervisionam os artefactos gerados. A adopção, contudo, ultrapassou o método. Uma Revisão Sistemática da Literatura realizada neste trabalho, seguindo as directrizes de Kitchenham sobre 646 artigos recolhidos, confirma benefícios substanciais a par de riscos recorrentes de fiabilidade, segurança, rastreabilidade e responsabilização, e identifica a ausência de orientação estruturada e repetível para integrar estas ferramentas em fluxos de trabalho profissionais como o principal problema por resolver na área. A evidência independente reforça este ponto: foi reportado que a maioria do código gerado por IA funcionalmente correcto contém defeitos de segurança, e que os estrangulamentos de entrega se deslocam para a revisão em vez de desaparecerem, indicando que o valor destas ferramentas depende do processo e não apenas da capacidade do modelo.

Esta dissertação aborda essa lacuna através da Design Science Research Methodology. Propõe-se uma framework orientada a processo para a colaboração Humano-IA governada, que trata os sistemas de IA como colaboradores de pleno direito cujas contribuições devem ser explicitamente delimitadas, validadas e rastreadas. A framework define fases do ciclo de vida com portões de qualidade específicos por fase, responsabilidades expressas como funções e não como cargos, e métricas de governação, ancoradas numa especificação persistente e versionada que mantém cada artefacto gerado ligado ao requisito que lhe deu origem. A framework é instanciada no Apex, uma aplicação web funcional que operacionaliza cada fase, portão e artefacto sobre uma ferramenta real de gestão de projectos, servindo de veículo de demonstração num contexto prático. A avaliação combina a aplicação do artefacto num projecto real, instrumentos padronizados de usabilidade percebida e carga cognitiva, entrevistas semiestruturadas com profissionais e especialistas em Agile, e uma avaliação analítica face a critérios de sucesso definidos. O contributo esperado é um modelo de processo fundamentado em evidência que permita às organizações adoptar assistência por IA sem abdicar do controlo sobre a

qualidade, a segurança e a responsabilização do software.

Palavras Chave

Colaboração Humano-IA; Ciclo de Vida de Desenvolvimento de Software; Modelos de Linguagem de Grande Dimensão; Inteligência Artificial Generativa; Framework de Processo; Governação de IA; Design Science Research; Desenvolvimento Orientado à Especificação; Vibe Coding; Engenharia de Software

Contents

List of Figures

List of Tables

List of Algorithms

Listings

Acronyms

AI	Artificial Intelligence
AI-DLC	AI-Driven Development Lifecycle
DRQ	Dissertation Research Question
DSRM	Design Science Research Methodology
GenAI	Generative Artificial Intelligence
LLM	Large Language Model
NASA-TLX	NASA Task Load Index
SDLC	Software Development Life Cycle
SLR	Systematic Literature Review
SUS	System Usability Scale

1

Introduction

Contents

The software engineering landscape is undergoing a structural change driven by the rapid evolution of Artificial Intelligence (AI). The rise of Generative Artificial Intelligence (GenAI) and Large Language Models (LLMs) has extended machine assistance beyond classification and retrieval towards the generation of coherent, context-aware artefacts, reshaping problem-solving and creative workflows across the technology sector. Within software engineering specifically, these technologies have introduced intent-driven programming, colloquially termed Vibe Coding, in which developers describe desired behaviour in natural language and iteratively refine what a model produces [1, 2].

This shift is not confined to writing code. LLMs now participate in requirements elicitation, architectural exploration, test generation, documentation, and debugging, touching every phase of the Software

Development Life Cycle (SDLC) [3–5]. The consequence is a change in the nature of the practitioner's work: from manually constructing software towards specifying intent, supervising generated artefacts, and remaining accountable for architectural soundness, correctness, and security.

1.1 Motivation

The economic pull towards adoption is strong and well documented, but the evidence on outcomes is considerably more nuanced than early enthusiasm suggested, and it is this gap between adoption and outcome that motivates this work.

Adoption is close to universal while trust is not, and the two are moving in opposite directions. The 2025 Stack Overflow Developer Survey reports 84% of developers using or planning to use AI tools, up from 76% the previous year, while the proportion trusting the accuracy of those tools fell from 40% to 29%; more respondents actively distrust AI output (46%) than trust it (33%) [6]. Industry telemetry places daily use higher still, at around 90% of developers [7].

That distrust is empirically warranted rather than merely cultural, and the decisive point is that functional correctness and security do not move together. Benchmarking agent-generated code on real-world tasks, Zhao et al. found that 61% of the solutions produced by a leading coding agent were functionally correct while only 10.5% were secure [8]. Assessment at larger scale agrees: across more than one hundred models, 45% of generated samples introduced an OWASP Top 10 vulnerability, rising to 72% for Java, and generated code contained roughly 2.74 times the vulnerabilities of human-written equivalents; critically, newer and larger models improved at producing functionally correct code while their security performance remained flat [9]. Capability alone therefore does not converge on secure output.

Productivity gains are also less automatic than assumed. Early controlled evidence was strongly favourable: a widely cited experiment reported developers completing an implementation task 55.8% faster with an AI assistant, although it remains unpublished in a peer-reviewed venue [10]. More recent work under different conditions reverses the sign. In a randomised controlled trial, 16 experienced open-source developers completing 246 tasks on their own mature repositories took 19% longer when permitted to use AI assistance, while estimating afterwards that it had made them 20% faster [11]. The contrast is itself informative: the favourable result was obtained on a self-contained task with no existing codebase to respect, the unfavourable one on mature repositories the participants already knew well. The same divergence appears at organisational scale: telemetry from more than ten thousand developers across 1,255 teams associates high AI adoption with 98% more pull requests merged but 91% longer review times, 154% larger pull requests, and no significant correlation with company-level delivery or quality outcomes [12]. The bottleneck moves downstream to validation; it does not disappear.

Several of these sources are industry-reported rather than peer-reviewed, and are identified as such wherever they are used. The argument here does not rest on the precision of any single figure, but on the consistency of the direction across independent sources and methods.

Taken together, these findings support a single interpretation that frames this dissertation: the value of AI assistance in software engineering is conditional on the process that surrounds it, rather than being a property of the model alone. Capability without governance redistributes effort rather than reducing it, and can degrade quality attributes that are expensive to recover later.

1.2 Research Problem

To characterise the state of practice rigorously rather than anecdotally, a Systematic Literature Review (SLR) was conducted following Kitchenham's guidelines, retrieving 646 articles and synthesising the resulting corpus around benefits, consequences, integration methodologies, stakeholder perspectives, and adopting industry sectors. The review is reported in full in Chapter 4.

Its central finding is methodological rather than technical. While the literature consistently reports meaningful benefits, it equally consistently reports that organisations lack established, clear methodologies to shape the responsible use of AI and LLMs within professional software development [13–16]. Proposed approaches remain fragmented across isolated tools and tasks, differ significantly in scope and level of abstraction, and share no common foundation regarding lifecycle integration, oversight practices, or evaluation criteria [3].

Established methodologies do not close this gap. Waterfall, Scrum, Kanban, and the Agile Manifesto delivered substantial improvements in adaptability and collaboration, but were designed for development environments in which every contributor is human [4]. They therefore lack explicit structures for governing AI participation, for bounding the context a model is permitted to act upon, for assessing model reliability, and for allocating responsibility over generated artefacts. Their iteration cadence also assumes that implementation is the binding constraint, an assumption the evidence in Section 1.1 directly contradicts.

The research problem addressed in this work is therefore the absence of structured, repeatable guidance and standardised evaluation practices for integrating AI and LLM-based tools across the SDLC while preserving human oversight, software quality and security, and organisational accountability.

1.3 Objectives and Research Questions

The objective of this dissertation is to design, instantiate, demonstrate, and evaluate a process-oriented framework that addresses the problem stated above. This objective is decomposed into the following research questions, which structure the design and evaluation of the artefact.

These are labelled Dissertation Research Question (DRQ) to keep them distinct from the review questions RQ1 to RQ5 of the systematic literature review in Chapter 4. The two sets do different work and are answered by different means: RQ1 to RQ5 ask what the existing literature reports, and are answered by synthesising the corpus; DRQ1 to DRQ4 ask what this dissertation's artefacts achieve, and are answered by designing, building, demonstrating, and evaluating them. Both appear in Table 6.1, which is precisely where the two could otherwise be confused.

1. **DRQ1:** How can AI-supported activities be explicitly mapped to SDLC phases so that the point, purpose, and boundary of AI participation are unambiguous?

2. **DRQ2:** Which governance mechanisms, quality gates, and human-in-the-loop practices are required to keep AI contributions traceable, validated, and accountable across the lifecycle?

3. **DRQ3:** To what extent can the framework's governance constructs be operationalised in a working software system, and which of them resist implementation?

4. **DRQ4:** Does applying the framework, through that implementation, provide practitioners with usable and cognitively sustainable support in a real development setting?

DRQ1 and DRQ2 are answered by the framework design in Chapter 6; DRQ3 by the construction of the reference implementation in Chapter 7, including the constructs that had to be weakened, substituted by a proxy, or left unimplemented; and DRQ4 by the demonstration and evaluation reported in Chapters 8 and 9.

1.4 Contributions

This dissertation makes four contributions:

1. **A systematic synthesis of the field.** A SLR over 646 retrieved articles, organising current evidence on the benefits, consequences, integration methodologies, stakeholder perspectives, and adopting sectors of AI and LLM use in software development, including a mapping of impacts across individual SDLC phases.

2. **A process framework for governed Human-AI collaboration.** A process-oriented framework that treats AI systems as first-class collaborators rather than as passive tooling, defining lifecycle phases, phase-specific quality gates, responsibilities expressed as functions rather than job titles, governance metrics, and a persistent versioned specification that keeps generated artefacts anchored to their originating requirements.

3. **A working reference implementation.** Apex, a split-stack web application that operationalises every phase, gate, and artefact of the framework against a real project management backend, demonstrating that the framework's governance constructs are implementable rather than merely describable.

4. **An empirical evaluation.** Application of the artefact in a real-world project, complemented by standardised measurement of perceived usability and cognitive workload, semi-structured interviews with practitioners and Agile experts, and an analytical assessment against defined success criteria.

1.5 Research Methodology

Two complementary methodologies are employed. The SLR, following Kitchenham's guidelines, establishes the evidence base and grounds the problem statement. The Design Science Research Methodology (DSRM) proposed by Peffers et al. structures the design, construction, demonstration, and evaluation of the artefact across its six activities. Chapter 2 describes both in detail and maps each DSRM activity onto the chapters of this document.

1.6 Document Structure

The remainder of this dissertation is organised as follows. Chapter 2 details the research methodologies. Chapter 3 presents the theoretical background on AI, the SDLC, and AI-assisted development practice. Chapter 4 reports the SLR in full. Chapter 5 formalises the research problem. Chapter 6 derives the research objectives from the review's findings and presents the proposed framework. Chapter 7 describes Apex, its reference implementation, including the design iterations driven by practitioner feedback. Chapter 8 reports the demonstration of the artefact in a real-world setting, and Chapter 9 its evaluation. Chapter 10 concludes with contributions, limitations, communication of results, and directions for future work.

2

Research Methodology

Contents

This chapter presents the research methodologies used in this dissertation, detailing the steps taken in each. The two methodologies are Systematic Literature Review (SLR) and Design Science Research Methodology (DSRM).

2.1 Systematic Literature Review (SLR)

The systematic literature review (SLR) conducted in this study follows the guidelines proposed by Kitchenham, who describes a systematic review as “a means of evaluating and interpreting all available research relevant to a particular research question, topic area, or phenomenon of interest.” According to Kitchenham, “systematic reviews aim to present a fair evaluation of a research topic by using a trustworthy, rigorous, and auditable methodology.” These principles ensure that the review process adopted in

this work is methodical, transparent, and replicable. The Kitchenham guidelines present three phases:

- **Planning the Review:** Define the motivation, research questions, selection criteria, and review protocol.
- **Conducting the Review:** Apply the protocol to identify and select relevant studies.
- **Reporting the Review:** Synthesize and communicate the findings from the selected literature.

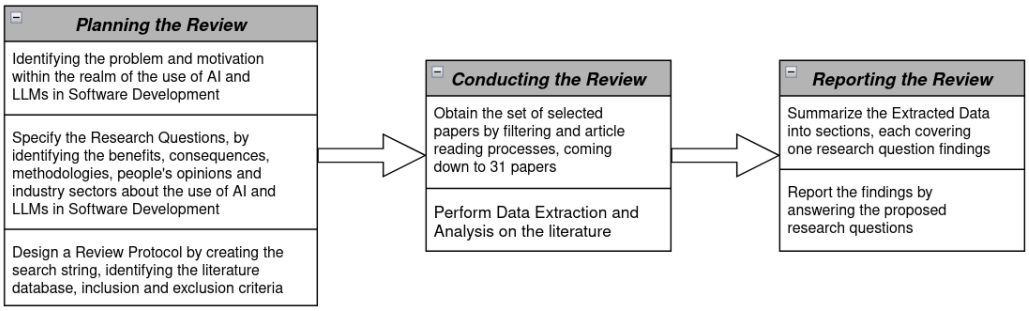

Figure 2.1: Three phases of the Systematic Literature Review

Figure 2.1 illustrates the three phases of the systematic literature review and the activities performed in each phase. The filtering process was conducted through multiple iterations: an initial screening of articles retrieved from the selected database, followed by an abstract-reading iteration to assess relevance, a filtering based on referenced works and Kitchenham's criteria using introductions and conclusions, and a final full-text review to determine the final set of studies. The SLR methodology was selected as it enables a structured and unbiased collection of existing research on AI-supported software development and its associated frameworks and management practices.s

2.2 Design Science Research Methodology (DSRM)

The Design Science Research Methodology (DSRM) focuses on the development of IT artifacts such as models, methods, or frameworks-that address identified organizational or technological problems. Unlike purely observational research, DSRM emphasizes the construction of solutions and the evaluation of their usefulness and applicability. Proposed by Peffers et al., the methodology follows a six-step process that supports systematic design, clear research contributions, and the generation of transferable scientific knowledge. This makes DSRM particularly suitable for research that proposes new frameworks or process-oriented solutions. The six steps are illustrated in Figure 2.2 and described below.

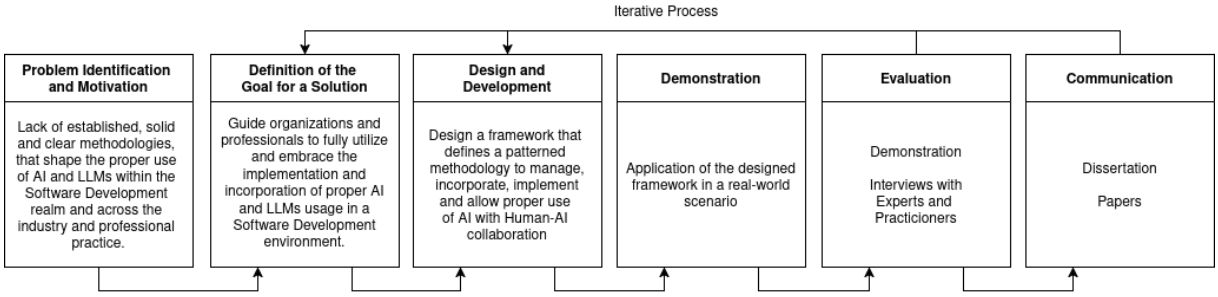

Figure 2.2: Phases of the Design Science Research Methodology

- **Problem Identification and Motivation:** This research addresses the lack of structured guidance, standardized methodologies, and evaluation practices for integrating AI and LLMs into software development while ensuring quality, accountability, and human oversight. This problem was identified through a Systematic Literature Review, which revealed fragmented practices and unclear human-AI role definitions across the SDLC. The motivation of this step is to map all AI related information to identify gaps informing both researchers and practitioners.

- **Definition of Objectives:** Based on the identified problem and the SLR findings, the research objectives aim to guide organizations in the effective, secure, and responsible adoption of AI and to maximize its potential use into an organized software environment through the definition of appropriate processes and governance mechanisms.

- **Design and Development:** Using the SLR findings as input, this step results in the design and construction of a process-oriented framework that defines phases, practices, roles, and decision points for human-AI collaboration across the SDLC.

- **Demonstration:** This step will consist of applying the proposed framework in a realistic organizational scenario with a consultancy operating with Scrum practices and AI-augmented development techniques. The framework will be instantiated across a limited number of short development iterations, illustrating how it guides AI tool usage, prompt formulation, human-AI collaboration, validation checkpoints, and responsibility allocation throughout the SDLC. The main outputs of this step are documented development artifacts, decision rationales, and observations that show how the framework addresses the identified problem in practice.

- **Evaluation:** The framework is applied in a realistic organizational scenario using Scrum-based and AI-augmented development practices, illustrating how it guides AI usage, collaboration, and validation throughout development iterations. Evidence from the demonstration and interviews will be synthesized to identify strengths, limitations, and opportunities for refinement, directly relating evaluation outcomes to the research objectives.

- **Communication:** This step will focus on disseminating the research results to both academic and professional audiences. This includes the preparation of the scientific article addressing the Systematic Literature Review, which will be submitted to the ACM Transactions on Software Engineering and Methodology journal; the submission of this report regarding the proposed framework along with the problem and its goals; and lastly the future development and submission of the final dissertation regarding the development of the framework.

2.3 Mapping the Methodology onto this Document

Because DSRM prescribes activities rather than document sections, Table 2.1 states explicitly where each activity is carried out in this dissertation. The mapping is not strictly linear: as Chapter 7 describes, feedback gathered during the construction of the reference implementation fed back into the design of the framework, which is the iteration path DSRM explicitly allows.

Table 2.1: Mapping between DSRM activities and the chapters of this dissertation

DSRM Activity	Chapter	Output
Problem Identification and Motivation	Chapters 4 and 5	Evidence base from the SLR; formalised research problem
Definition of Objectives	Chapter 6	Objectives derived from the review findings
Design and Development	Chapters 6 and 7	The framework, and Apex as its reference implementation
Demonstration	Chapter 8	Application of the artefact in a real-world project
Evaluation	Chapter 9	Usability and workload measurement, interviews, analytical assessment
Communication	Chapter 10	Publications and dissertation defence

3

Research Background

Contents

This chapter presents the theoretical background of the topics associated with the research. The topics covered are Artificial Intelligence, the Software Development Life Cycle, AI in Software Development, Vibe Coding, and Spec-Driven Development.

3.1 Artificial Intelligence

Artificial Intelligence (AI) refers to systems capable of performing tasks that traditionally require human cognitive abilities, such as reasoning, decision-making, pattern recognition, or language understanding. Early AI systems followed rule-based approaches, relying on explicitly defined logic to guide decision-

making. More recent developments emphasize data-driven and statistical learning, where models improve their performance by identifying patterns in large datasets. AI now underpins a wide range of applications including recommendation systems, natural language processing, predictive analytics, and autonomous systems, reflecting its expanding relevance across different industries.

Generative Artificial Intelligence (GenAI) refers to a subset of AI systems designed to create new content, rather than classify or retrieve existing information. These systems generate text, images, audio, or code based on learned patterns in their training data. GenAI models operate by predicting likely future outputs given prior context, allowing them to produce coherent and contextually relevant content. This generative capacity has positioned GenAI as a significant innovation in domains that rely on iteration, prototyping, and creative exploration, including software development.

Large Language Models (LLMs) are a specific class of GenAI systems trained on vast corpora of text and source code. Based on transformer architectures, LLMs are capable of processing and generating sequences with contextual awareness. Their primary function is next-token prediction, but this capability enables more complex tasks such as summarization, explanation, reasoning, conversational interaction, and code generation. Within software engineering, LLMs are relevant because they can interpret programming intent expressed in natural language and generate syntactically valid code in a variety of programming languages. They are also able to explain, refactor, or restructure existing code, as well as produce tests, documentation, and other boilerplate artifacts with far less manual effort. These capabilities position LLMs as collaborative tools that can support both novice and experienced developers throughout different stages of software development. [1–5, 13–39]

3.2 Software Development Life Cycle

The Software Development Life Cycle (SDLC) describes the structured phases involved in building and maintaining software systems. Therefore, the SDLC is commonly structured into six main phases: requirements analysis, where system functionality is identified through requirements engineering; design, which defines the system architecture, data structures, and component interactions; implementation, focused on writing and integrating code; testing, where correctness, reliability, and performance are verified; deployment, which delivers the system to end users; and maintenance, involving ongoing updates, improvements, and operational support throughout the system's lifetime.

The SDLC is central to this research because LLMs can engage differently across its stages. For example, LLMs can support implementation by generating code, testing by producing test cases, and maintenance by summarizing or refactoring existing code. However, the developer remains responsible for architectural decisions, validation, integration, and quality assurance.

Software development processes have evolved to emphasize iterative delivery and continuous inte-

gration, supported by methodologies such as Agile and DevOps. Despite these improvements, many development tasks remain manual, repetitive, and time-intensive. Developers spend significant time debugging, documenting code, maintaining consistency across repositories, and handling routine implementation tasks. As systems scale in complexity, these challenges become more pronounced, creating pressure to increase productivity without compromising maintainability or security. Organizations therefore seek tools that can automate or streamline specific tasks while maintaining developer oversight. This ongoing search for efficiency is a key factor driving interest in AI-assisted development workflows. [3, 4, 26, 35]

3.3 AI in Software Development

Advances in LLMs have introduced new forms of interaction between developers and software tooling. In what is often described as conversational or intent-driven programming, developers describe desired functionality in natural language and iteratively refine code generated by the model. This pattern represents a shift from writing code directly to supervising and guiding automated generation. Studies indicate that LLMs can accelerate prototyping, clarify unfamiliar codebases, and help maintain documentation quality. These capabilities can be especially valuable in early-stage development or when exploring new frameworks and libraries.

The integration of AI into the development process is increasingly characterized as collaborative. Rather than fully automating software creation, LLMs function as tools that support developers in tasks involving pattern recognition, summarization, or structural reasoning. The developer remains responsible for architectural decisions, validation, integration, and ensuring alignment with project requirements.

This collaborative model suggests a shift in the nature of developer work: from manually constructing code to curating and verifying generated artifacts. Research in this area focuses on how workflows adapt, how developers form trust in AI outputs, and how responsibilities are distributed between human judgment and automated generation. These themes are further examined in Chapter 4. [3, 4, 13, 14, 22, 26, 28, 35]

3.4 Vibe Coding

Vibe Coding refers to a style of software development in which developers describe their intentions, requirements, or constraints in natural language, and a generative model produces corresponding code or related artifacts. In this approach, the developer does not begin by writing syntactically precise code. Instead, development proceeds through a conversational interaction, where the model proposes implementations that are then reviewed and refined by the developer. This interaction model builds on the

idea of human-AI collaboration introduced in the following sections. While the model generates initial drafts or prototypes, the developer evaluates correctness, aligns the output with project requirements, and iteratively modifies the prompt or the produced code.

Vibe Coding shifts part of the programming effort from manual construction to specifying intent and assessing alternatives, which can influence how tasks are conceptualized and distributed during development. The approach does not replace traditional programming but reframes how initial structure and exploratory code are produced. As such, it has implications for developer workflows, cognitive load, and the distribution of effort across the SDLC. These implications, including reported advantages and limitations, are examined in detail in the Systematic Literature Review reported in Chapter 4. [1, 2]

3.5 Spec-Driven Development

Spec-driven development is an emerging engineering practice in which a persistent, versioned specification, rather than the source code itself, is treated as the primary artefact that both developers and AI agents work against. Instead of prompting a model directly against a codebase and accepting whatever it produces, the practice interposes a written specification, of requirements, design decisions, and constraints, that the model must consult and satisfy before code is generated, and that is kept in sync with the codebase as it evolves. Industry examples of this pattern include AWS Kiro's three-stage requirements, design, and tasks workflow, GitHub's Spec-Kit, which adds a "constitution" of immutable project principles that generated code may not violate, and the Tessl Framework, in which, as its own materials put it, the specification itself becomes the maintained artefact rather than the code [40].

The practice is tracked by Thoughtworks' Technology Radar as an emerging AI-assisted engineering technique, named specifically "spec-driven development" [40], and has begun to receive academic attention: Farrag frames it as specification-driven governance, arguing that it addresses what is termed the productivity-reliability paradox in AI-augmented development, though this framing is currently reported only in a preprint that has not yet completed peer review [41]. The practice responds directly to the risk profile characterised in Section 1.1: narrowing and anchoring the context a model is permitted to act upon is a governance mechanism, not merely a convenience, since it gives generated output a stable reference to be validated against rather than leaving correctness to be inferred from the prompt alone. This precedent is the direct grounding for Spec-Anchored Continuity, the persistent specification mechanism at the centre of the framework proposed in Chapter 6.

4

Systematic Literature Review

Contents

In this section, the execution process of the systematic literature review is described. The description follows the SLR stages, which are as follows: Planning, Conducting, and Reporting the results.

4.1 Planning the Review

This section details the planning stage of the SLR methodology. The motivation for the review is presented, then the research questions addressed in the search, and lastly the search method employed.

4.1.1 Motivation

The integration of AI, particularly LLMs and GenAI, represents a significant transformation in Software Engineering. These technologies are increasingly adopted across the SDLC to support activities such as code generation and completion, documentation, requirements engineering, automated bug fixing, and developer learning. Their use has been associated with notable productivity gains, improved efficiency, and enhanced support for complex development tasks by augmenting traditional development tools and information retrieval practices [3, 4, 13, 14, 22, 26, 28, 35].

Despite these benefits, the rapid adoption of AI and LLMs in software development has introduced unresolved challenges that motivate systematic investigation. The literature highlights persistent concerns related to the quality, reliability, and trustworthiness of AI-generated artifacts, as well as technical and methodological barriers such as limited explainability and the black-box nature of advanced models. In addition, effective utilization of LLMs remains highly dependent on prompt formulation, while existing research often focuses on isolated tasks, leaving several SDLC phase,-particularly requirements engineering and design, with limited practical guidance.

Therefore, the motivation for this Systematic Literature Review is to comprehensively map and synthesize existing research on the use of AI and LLMs in Software Development. By organizing current evidence on benefits, limitations, methodologies, stakeholder perspectives, and application domains, this review aims to identify research gaps and inform both researchers and practitioners on how to better leverage AI technologies across the SDLC.

4.1.2 Research Questions

At the start of the Planning the Review section of this SLR, there were a lot of research questions that could have been written about this overly general topic of AI adoption in the development process of applications. After making a throughput selection of the Research Questions that were more relevant and specific about the theme the Chosen RQs pool became a lot more concise for this Systematic Literature Review. As such, this systematic review plans to answer the following RQs:

Table 4.1: Research Questions and Focus Areas

RQ	Research Question	Focus
RQ1	What are the benefits of using and incorporating Vibe Coding, AI, and Large Language Models in software development?	Identify and categorize improvements in productivity, creativity, code quality, automation, collaboration, and innovation enabled by AI-driven tools.

RQ2	What are the consequences that may arise from using and incorporating Vibe Coding, AI, and LLMs in software development, both in code generation and other sections?	Analyze risks such as technical debt, bias, reduced developer learning, code ownership, ethical issues, and dependency on AI tools.
RQ3	What methodologies and management methods are being used to integrate Vibe Coding, AI, and LLMs into software engineering processes and workflows?	Explore frameworks, best practices, and process models (e.g., AI-augmented Agile, human-in-the-loop development, prompt engineering practices).
RQ4	What are developers' and other stakeholders' opinions regarding the use of AI and LLMs in software development?	Assess perceptions, trust, satisfaction, and acceptance of AI-assisted coding among developers and organizations.
RQ5	Which industry sectors are adopting AI for application development, and how do they benefit from its integration?	Map adoption trends across industries (e.g., finance, healthcare, IT, education), highlighting sector-specific motivations and outcomes.

4.1.3 Review Protocol

The systematic literature review starts with a literature search. A search string is defined to then be used in the data source of choice. The search criteria used for this review resulted in 646 articles to review.

Table 4.2: Research Elements and Details

Element	Research Details
Source	EBSCO
Final Search String	(“Vibe Cod*” OR “AI models” OR “AI agen*” OR “large language model*” OR “generative AI” OR openai OR “chatgpt” OR LLM* OR “claude” OR “gemini” OR copilot OR co-pilot) AND (“software development” OR “application development” OR “code generation”)
Search Method	Search string found in the abstract of articles in English academic journals or conference materials, with no date range limit
Results	646

The aim of this review was to research the use of AI/LLMs in developing applications; but that specific topic can be written in many different ways. When talking about this theme, a very recent term used in the world of Software Development came to mind: "Vibe Coding". For such a new topic and specific term regarding the use and adoption of AI in the development process of software, there were used

a lot of keywords in the search string that have the same significance and could replace it since the topic was recent but its methodology wasn't, it was still used but with different expressions and the term "Vibe Coding" itself didn't provide a lot of relevant articles covering the theme. Keywords such as "code generation" and "application development" used in conjunction with "generative AI" and "AI models" provided a search string that could get the desired world of use and adoption of AI and LLMs in the software development process as a whole.

The defined search string was then used to query the EBSCO Online Digital Library. For this review, the "Advanced Search" option was used and the query was executed in the AB abstract field and title of the papers. To limit the number of articles found, the search was limited to articles in academic journals or conference materials that had the full text available and were in English.

4.2 Conducting the Review

This section covers the second stage of the SLR. To conduct the review, the 646 articles extracted in the previous stage were analyzed according to the criteria used for the filtering process. The criteria used are detailed and a representation of the results found along the process is provided.

4.2.1 Filtering Process

The filtering process starts with the 646 articles that resulted from the search string query and Figure 4.1 shows that exact process. Extracting and downloading those 646 articles was impossible due to EBSCO's web page loading capacity which only allowed the download of 50 articles at a time; after reloading and extending the page to its maximum, the website could only download and display, by relevancy order, 467 out of the 646 articles obtained from the search string.

To organize the articles and to help the filtering process, the AI-Powered Systematic Review Management Platform "Rayyan" was used. From the 467 extracted, 227 duplicates were identified and removed, resulting in 350 articles left for Abstract Reading. The abstract of the resulting articles was read to ascertain the topic covered and 286 articles were rejected due to the reason of covering another topic other the overall theme of "Developing applications using AI" leaving exactly 64 articles.

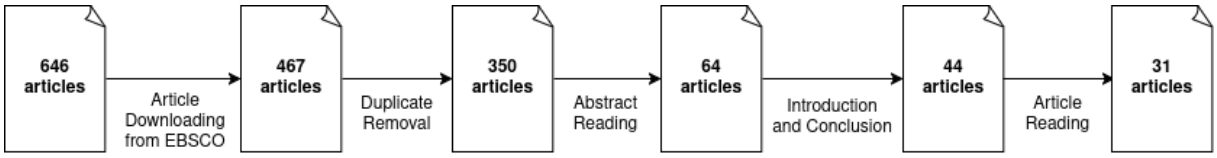

Figure 4.1: Articles Filtering Process

After making sure that all the relevant papers concerning the theme were still being considered, it

was time to properly read the content of these papers, and during this section these 64 articles were separated into 4 categories: Included, Maybe, Excluded and Skipped.

The "Skipped" articles, although they passed through the abstract reading phase, after reading the introduction and conclusion of those papers and evaluating them on regards if they are relevant to the theme, if they are relatively recent (which is important due to the highly dynamic and fast-evolving nature of the field) and if any of the five research questions were answered, there were 20 skipped papers and properly excluded. The number of "Maybe" articles was always changing during the article reading section, but the files that were classified as "Maybe" was due to not knowing if its content would be relevant to the theme in question. Most of them were properly excluded due to prioritizing more recent studies to reflect the current state of the art and the papers that fit into the topic were included.

If the reviewed sections of an article provided a clear and explicit answer to any of the five research questions, the article was included in the study. However, when the information presented was unclear, insufficient, or lacked the necessary detail to confirm whether an answer was provided, the predefined inclusion and exclusion criteria were applied to determine eligibility. These criteria are summarized in Table 4.3. From the initial set of 64 articles, after excluding and deciding which "Maybe" articles were relevant, 31 met the criteria and were retained for analysis.

Table 4.3: Inclusion and Exclusion Criteria

Inclusion Criteria	Exclusion Criteria
• Relevance regarding AI and LLM usage in the development of software applications • Relevance regarding benefits and consequences of incorporating and using AI and LLMs in the development of software applications • Relevance regarding methodologies and management methods used to integrate AI and LLMs into software engineering processes and workflows • Relevance regarding people's opinions with AI and LLMs use in software development • Relevance regarding which industry sectors are adopting AI for application development and its benefits	• Not written in English • Does not clarify any of the Inclusion Criteria • Outdated due to the highly dynamic nature of the field (anything older than 2024)

4.2.2 Data Extraction Analysis

Having selected the articles, these can now be analyzed, subdivided according to the venue in which they were published, journal or conference material, as well as identifying the conferences and journals used and the year it was published. Curiously enough, none of the selected papers were categorized as Conference Materials, only Peer Reviewed Academic Journals categorized as such by EBSCO were selected and extracted as shown in Figure 4.2.

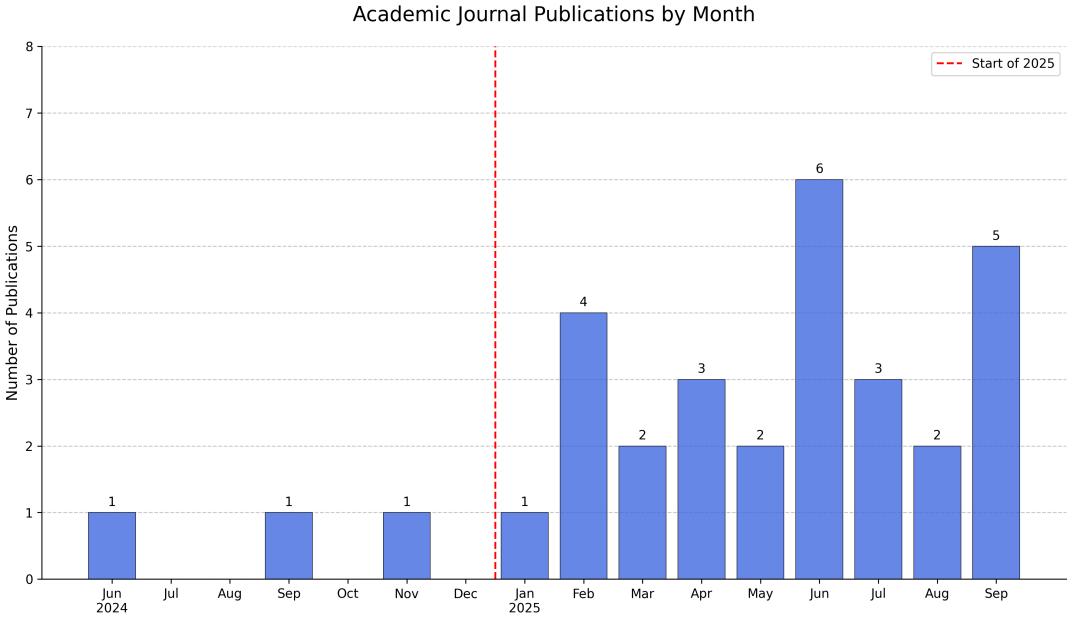

Figure 4.2: Academic Journal Publications

4.3 Reporting the Review

This section covers the third and last phase of the SLR methodology. The results from the analysis of each selected article and the collected information relevant to the review are presented here, which will make possible to answer the Research Questions defined earlier. The information is then summarized according to each of the determined research questions.

Table 4.4: Journals and Publication Counts

Journals	Nr. Publications
ACM Transactions on Software Engineering & Methodology	10
Information and Software Technology	3

Continued on next page

Journals	Nr. Publications
Advanced Science	1
Applied Computer Systems	1
ARO-The Scientific Journal of Koya University	1
Array	1
BenchCouncil Transactions on Benchmarks, Standards & Evaluations	1
BioData Mining	1
E3S Web of Conferences	1
Engineering Applications of Artificial Intelligence	1
Expert Systems With Applications	1
Frontiers in Computer Science	1
IEEE Access	1
Informatica Economica	1
Information Systems & e-Business Management	1
Journal of Computer Information Systems	1
Journal of Software: Evolution & Process	1
Neurocomputing	1
Technology Audit & Production Reserves	1
Tehnički Glasnik	1

4.3.1 Benefits of AI and LLMs usage in Software Development

The first research question looks into the benefits of using and incorporating AI and Large Language Models in software development to developers, companies and other stakeholders. The benefits found in the literature have been collected in Table 4.5. To answer this question, this section will analyze the benefits found.

Table 4.5: Benefits of using AI and LLMs in Software Development

Benefits	Description	#	References
Improved Code Quality, Accuracy, and Traceability	"In practice, it has been proven that using Generative AI can increase efficiency and productivity by improving code quality and optimizing work with faster times." [14].	20	[3, 4, 13–16, 18, 19, 21–24, 26, 27, 29–31, 33, 34, 36]

Continued on next page

Benefits	Description	#	References
Increased Productivity and Efficiency	"Accelerates the coding process by converting natural language or abstract research intent (a vibe) into functioning software modules, dramatically shortening development cycles" [1].	18	[1–4, 13, 14, 19, 20, 24, 27, 29, 30, 33–37, 39]
Automation of Testing, Debugging, and Repair	AI and LLMs "can increase the efficiency of software development by automating coding, bug detection, debugging, and personalizing software according to user needs" [14].	12	[1, 3, 16, 17, 20,21,29,30, 33–35, 37]
Optimization of Code Performance and Efficiency	"The proportionally small number of studies in SE subfields like Build, Release, Deploy, Operate and Monitor indicates a broad untapped area for cost savings and novelty, which could bring major optimisation in Software Engineering" [28].	10	[1, 2, 4, 13, 14,17,24,28, 33, 35]
Support for Early Software Development Phases (Requirements & Design)	AI and LLMs have effectiveness in "...enhancing stakeholder communication and requirement documentation" while noting challenges with bias and accuracy in requirements and design elicitation [29].	8	[4,15,16,22, 27,29,33,35]
Automated Documentation and Summarization	"LLMs can [...] automatically generate documentation for codebases, including explaining the purpose and functionality of specific functions, classes, and modules. This ensures that documentation stays current and can be updated as code evolves, providing developers with easily understandable, well-structured explanations of how various parts of the system work." [24].	8	[4,13,17,22, 24,26,35,36]
Enhanced Security and Vulnerability Management	Gen AI technology good practices enhances SDLC security by supporting development through "vulnerability detection, threat modeling, secure coding practices, and incident response" [17].	6	[17, 18, 26, 33, 35, 39]
Knowledge Acquisition and Understanding	"Reduce technical barriers and transform the programming process into a collaborative, conversational, and more accessible act" [2].	5	[2,13,20,27, 33]
Management and Organizational Support	Assist in managing and "curating organizational information about day-to-day organizational practices" [27].	5	[3, 4, 16, 17, 22]

Continued on next page

Benefits	Description	#	References
Developer Well-being and Satisfaction	AI tools improve developer satisfaction by enabling developers to "prioritize interesting and fun tasks through automating menial, boring tasks" [27].	4	[4,19,27,33]
Democratization and Accessibility	AI integration aims to "shift the programming paradigm from a traditional code-centric approach to one that involves verbal interactions between the programmer and the artificial intelligence agent being used", making programming more accessible [2].	2	[1,2]

The integration of AI and LLMs into software development yields profound benefits, foremost among which is the dual improvement of productivity and code quality, which appear to be mutually reinforcing and more abundant within the articles. This acceleration does not come at the cost of quality; on the contrary, the literature indicates that AI helps in generating code that adheres to high standards, which directly increases the quality, accuracy, and particularly the traceability of the final product. [30]. This synergy is further supported by the automation of traditionally time-consuming quality assurance tasks like testing, debugging, and repair, where AI's ability to refine implementations and correct errors significantly reduces manual intervention and boosts the code's overall robustness; specifically, LLMs leverage explicit feedback, such as compiler error messages, in follow-up prompts to iteratively fix their own coding errors within the same conversation context.

Beyond the direct impact on the codebase, the integration of AI yields substantial benefits for the developers themselves by improving job satisfaction and democratizing programming knowledge. Although being the benefit that is mentioned less in the reports, its equally important to the art. By automating monotonous and menial tasks, AI tools allow developers to prioritize more engaging and intellectually stimulating work that requires professional judgment, which in turn enhances their well-being and satisfaction. Concurrently, AI acts as a catalyst for knowledge acquisition and collaboration. It lowers technical barriers and helps transform the programming process into a more conversational and accessible activity, making it easier for developers to understand complex codebases. This evolution fosters a more inclusive environment, shifting programming from a purely code-centric discipline to an interactive one between the programmer and an intelligent agent, thus making it more accessible to a broader audience.

The advantages of incorporating AI and LLMs are not confined to a single phase but extend across the entire SDLC. In the crucial early stages (Requirements Analysis and Design Modeling), these technologies enhance stakeholder communication and aid in the elicitation and documentation of requirements and design specifications, for example, by transforming user stories into formal models like UML diagrams [15]. The benefits continue into the implementation and maintenance phases, with AI tools

specifically designed to optimize the computational performance of generated code, improving its runtime and memory usage, often exceeding typical human performance. Furthermore, AI provides critical support for non-functional requirements, most notably through enhanced security and vulnerability management via support for threat modeling and secure coding practices. Finally, the persistent challenge of documentation is addressed by LLMs that can act as summarizers to automatically generate multi-intent comments and other documentation artifacts, ensuring the long-term maintainability of the software.

4.3.2 Consequences of AI and LLMs usage in Software Development

This second section will analyze, cover and show more about the consequences that may arise while using and incorporating AI and Large Language Models in Software Development both in code generation and other sections of the art. The consequences found in the many articles have been collected in Table 4.6.

Table 4.6: Consequences of using AI and LLMs in Software Development

Consequences	Description	#	References
Increased Operational Costs and Resource Constraints	"The financial burden can become substantial and must be considered when LLMs are deployed to handle a large number of tasks" [5].	20	[3, 5, 20, 22–24, 26, 28, 35, 38]
Reliability, Accuracy, and Hallucination Risks	This is particularly problematic in complex or niche scenarios. "Because this AI can simply make things up or sometimes spit out things that don't even exist" [13].	15	[2, 4, 15, 16, 18–20, 22, 23, 30, 31, 34–37]
Security Vulnerabilities in Generated Code	"Recent studies suggest that code generated by AI models often contains security vulnerabilities that may follow patterns different from those in human-written code. Evidence suggests that AI-generated code may harbor unique security risks in areas such as cryptographic implementation, memory management, and error handling" [39].	15	[3, 13, 14, 16–20, 24, 26, 28, 33, 36, 37, 39]
Ethical, Legal, and Intellectual Property Issues	"Questions about the ownership and licensing of that code will arise, particularly when the models are trained on publicly available, open-source projects" [24].	8	[2, 3, 13, 17, 19, 22, 24, 35]

Continued on next page

Consequences	Description	#	References
Homogeneity and Bias Propagation	As LLMs increasingly train on LLM-generated code, there is a risk of code homogeneity, reducing diversity in coding practices and potentially perpetuating slow, biased, or insecure patterns. This reduces the diversity of code practices over time, as LLMs tend to "reaffirm patterns seen in their training data (e.g., public GitHub repositories)" [35].	8	[3,13,14,17, 19,24,34,35]
Evaluation and Standardization Challenges	"It is urgent to establish an effective, objective, and complete evaluation system for large language models for code" [3].	7	[3,14,15,23, 26,32,39]
Risk of Over-reliance and Cognitive Decline	"Of course, sometimes sometimes there is almost the danger that you stop thinking for yourself a little. I think that might also be a problem in the future, that if AI is constantly available, people will tend to become lazy and stop thinking for themselves" [13].	4	[2,13,19,27]
New Overhead and Workflow Inefficiencies (Human Review)	Integrating AI requires developers to shift their focus, often spending more time reviewing, verifying, and refining outputs, rather than writing new code because LLMs still "require human oversight and review of their outputs before implementation" [29].	4	[2,13,22,29]
Ineffectiveness of Existing Security Tools	Existing tools detect only "46.7% of vulnerabilities in AI-generated code, a significantly lower rate than for human-written code vulnerabilities (66.4%)" [39].	3	[17,26,39]

The integration of GenAI and LLMs into software development introduces significant technical, ethical, and operational consequences that demand careful mitigation and human oversight with operational and technical risks being the most frequently cited in the literature, as detailed in Table 4.6. Operational efficiency is threatened by increased costs and resource constraints. The computational and financial costs for training, fine-tuning, and deploying these large-scale models can be substantially higher than traditional methods, potentially offsetting the anticipated productivity gains.

Another major technical concern revolves around the inherent limitations of these models concerning reliability, accuracy, and security which was the second most referenced consequence in the literature. LLMs are fallible and frequently produce erroneous outputs, commonly referred to as "hallucinations", which involve the generation of plausible but incorrect or fabricated information. This is particularly problematic when tackling complex, niche or specialized programming scenarios. Furthermore, current security analysis tools struggle to detect these unique AI-generated vulnerabilities effectively, only identifying about 46.7% of issues in AI code compared to 66.4% in human-written code, suggesting a

fundamental mismatch between existing frameworks and the new types of flaws introduced by AI [39]. The resultant volume of potentially low-quality or insecure AI-generated code necessitates implementing scalable and specialized testing and quality assurance processes to maintain the integrity of software systems. Although being mentioned less in the articles, this consequence is equally important as the other consequences mentioned in the literature and ties directly with the the prevalence of the Reliability, Accuracy, and Hallucination Risks with AI and LLMs .

Beyond the technical domain, the rapid adoption of AI introduces profound risks related to human factors, ethics, and economic viability. A primary worry is the heightened risk of over-reliance on AI, which may lead to a reduction in developers' critical thinking skills, independence, and overall software engineering expertise. Developers, accustomed to instant assistance, may become reluctant to engage in complex coding or troubleshooting, resulting in a gradual loss of core development skills. Ethical challenges include the potential for biases embedded within training data to be perpetuated and magnified in generated code, particularly critical in sensitive industries [24]. Furthermore, legal and intellectual property issues pose major obstacles, as it remains unclear who owns the code generated by AI models and how to ensure compliance with licensing requirements, especially when models utilize open-source code for training [35].

These challenges highlight the essential and continuing role of human developers, especially in performing rigorous verification and maintaining quality throughout the SDLC. The inherent non-deterministic nature of LLMs, where the same prompt can yield different outputs, coupled with high output variability, mandates extensive human oversight and review before integrating AI-generated artifacts into production. Developers must account for the time spent correcting inaccuracies or mitigating unexpected vulnerabilities, which can counteract the intended efficiency gains. The process of prompting and refining AI outputs may require more time than anticipated. Moreover, the lack of standardized benchmarks and evaluation metrics makes it challenging to objectively assess the genuine contribution of AI tools to quality assurance and security within the SDLC, hindering accurate comparison and validation of their effectiveness. Therefore, successful integration requires not only highly effective prompting strategies but also specialized tools and methodologies designed to manage the complexity and risks associated with auto-generated code and support within a SDLC professional environment.

4.3.3 Methodologies and management methods used to integrate AI and LLMs in Software Development

This section will aim to answer to the third research question and analyze the literature in order to list all the methodologies and management methods used to integrate AI and LLMs in Software Development.

Table 4.7: Methodologies and Management Methods used to integrate AI and LLMs in Software Development

Methodologies	Description	#	References
In-Context Learning (ICL) / Few-shot Learning	"ICL seeks to improve the abilities of LLMs by integrating context-specific information, in the form of an input prompt or instruction template, during the inference and thus without the need to perform gradient-based training" [38].	9	[15, 16, 20, 22, 26, 33–35, 38]
Prompt Engineering (PE) / Prompt-Driven Development	"The effectiveness of prompt engineering critically determines the performance of leveraging the capabilities of LLMs". [26]	5	[3,20,26,32, 33]
Chain-of-Thought (CoT) Prompting	"A CoT is several intermediate natural language reasoning steps that lead to the final answer... the CoT only uses natural language to describe how to solve a problem step by step sequentially". [31]	5	[3,16,20,26, 31]
Human-In-The-Loop (HITL) Paradigm	"While overall, LLMs exhibit decent reliability in their outputs with varying degrees, they still require human oversight and review of their outputs before implementation". "The importance of the human contribution in identifying and resolving problems to maintain software quality is widely acknowledged by many authors, which also gives rise to the Human-in-the-Loop (HITL) paradigm" [29].	5	[15, 24, 29, 35,37]
Structured Prompting / Structured Chain-of-Thought (SCoT)	"SCoT denotes several intermediate reasoning steps constructed by programming structures", and "SCoT prompting asks LLMs first to generate an SCoT and then output the final code." [31]	4	[23, 26, 30, 31]
Multi-Agent Systems / Self-Collaboration Approach	"Different agents play different roles and collaborate to fulfill user requirements". "[...] a self-collaboration framework with role instruction, which allows LLM agents to collaborate with each other to generate code for complex requirements" [21,37].	4	[13, 18, 21, 37]
Integration with External Tools and Domain Knowledge	"Combining the linguistic capability of LLMs with specialized domain knowledge to enhance output quality, functionality, or security". "The integration of relevant context into prompts may further improve current GenAI models, thereby reducing hallucinations and increasing reliability". [13,37]	3	[3,4,13,37]

Continued on next page

Methodologies	Description	#	References
AI-Assisted Pair Programming / Human Oversight	"LLMs still require human oversight and review of their outputs before implementations, and they present unique characteristics to be considered before adoption". "The AI orchestrator will coordinate the perception and action of each agent" [29, 35].	3	[2, 29, 35]
Retrieval-Augmented Generation (RAG)	"RAG have gained attention. ICL and RAG offer a low-computation option by eliminating the need for parameter updates". "RAG demonstrates higher effectiveness than ICL but falls short of LoRA's effectiveness across all three CodeLlama model variants". [38]	2	[20, 38]
AI-Augmented Agile Development (e.g., AgileGen Framework)	"[...] we propose a human-agent collaborative generative software development approach, focusing on human involvement at the beginning (scenario decision-making) and end (acceptance and recommendation decision-making) of each iteration" [4].	1	[4]
Utilizing Gherkin Language (Behavior-Driven Testing)	"[...] we use Gherkin to supplement unclear user requirements with well-defined acceptance criteria, guiding the generation of applications that align with user requirements" [4].	1	[4]
Vibe Coding	"[...] emerging concept of vibe coding, in which artificial intelligence (AI) accelerates the coding process by converting natural language or abstract research intent (a vibe) into functioning software modules" [1]	1	[1]
Integration of LLMs with Traditional Testing	"the integration of LLMs with traditional approaches has emerged as an important direction for future exploration". "By combining them together, there can be 1.4× to 2.3× improvement than the baselines". [37]	1	[37]

The integration of LLMs and GenAI into software engineering has necessitated the creation of structured methodologies and management paradigms focused on optimizing model performance, defining human-AI collaboration, and formalizing the Software Development Lifecycle itself. This analysis section details the core integration methods, ranging from subtle parameter tuning to large-scale, process-oriented frameworks.

The fundamental layer of integration, and a clear focus of the literature, relies on methods to improve model interaction by prompts. This is led by In-Context Learning (ICL), which, with 9 articles, is the most frequently discussed technique, seeking to improve LLM abilities by integrating context-specific

information during inference.

Closely related is Prompt Engineering (PE), recognized as the practice of designing inputs to elicit reasoning and knowledge from LLMs for specific tasks. The effectiveness of LLMs is critically determined by the quality of these prompts. This principle led to the adoption of advanced reasoning techniques like Chain-of-Thought (CoT) prompting, where the model generates intermediate natural language steps leading to the final answer and building significantly upon CoT is Structured Chain-of-Thought (SCoT), another prompting technique that explicitly introduces programming structures (such as sequential, branch, and loop structures) into the reasoning steps. [31]

Given the inherent limitations of LLMs, including their overall output reliability and high variability, the Human-In-The-Loop (HITL) paradigm is considered mandatory for maintaining software quality [29]. HITL ensures human developers maintain critical oversight by reviewing and verifying AI-generated outputs before they are implemented, recognizing the continued necessity of human contribution in resolving problems. This human-AI teamwork approach is formalized in frameworks such as AI-Augmented Agile Development (AgileGen) which is mentioned once [4], which structures collaboration into lightweight iterations and reserves key human decision-making at the beginning (scenario definition) and end (acceptance and recommendation) of each cycle. This management strategy prevents the accumulation and propagation of errors that can occur in fully automated multi-agent systems. The one thing that is prevalent and that should always be a non-negotiable, is the presence of a developer that is always verifying AI-generated outputs before they are implemented to always ensure good and secure quality code and development throughout the whole Software Development Lifecycle.

Finally, "Vibe Coding" which is mentioned only once in the literature, and ties directly with the use and adoption of AI in the development process of software. This paradigm represents a conceptual leap in prompt application, accelerating the development process, particularly for domain-specific tasks like biomedical software and other sectors in the industry, by translating abstract intent and natural language directly into functional code and applications. Although Vibe Coding is relatively recent and is not mentioned a lot in the literature, its becoming the new "trend" and its the evolution of prompt based development, however its important to say that "Vibe Coding" has issues and a lot of problems, translating these "vibes" reliably requires strict use of methodologies and methods like the ones that were collected in this review to ensure better results [1].

4.3.4 Developers' and other stakeholders' opinions regarding the use of AI and LLMs in Software Development

Following the analysis of the technical benefits, consequences, and integration methodologies, it is equally crucial to understand the human perspective. This section will address the fourth research question, analyzing the literature to explore the opinions and sentiments of developers and other stake-

holders regarding the broader use of AI and LLMs in software development. Understanding the reception and concerns of the people actively using these tools is essential for a complete picture of their impact, and the findings of the revision of the literature will be covered in Tables 4.8 to 4.10.

Table 4.8: Public / Researchers Opinions on AI and LLMs in Software Development

Opinion Category	Description	#	References
General Trust / Perception	People's perception of systems powered by artificial intelligence as a mixture of "software and sorcery." [3]	6	[2, 3, 18, 19, 32, 34]
Misinformation Risk	"Users highlight its potential to misguide or hallucinate to students." [19]	4	[13, 19, 22, 26]
Bias and Impartiality Concerns	Users express dissatisfaction over perceived "leftist bias and propaganda in ChatGPT." [19]	3	[3, 19, 34]
Output Diversity / Similarity	"57% similarity overall with 43% variability, presenting that each model has its unique characteristics." [29]	2	[4, 29]
Legal, Copyright, and Licensing Issues	"Questions also surround the copyrightability/ownership of generated code [...]" [3]	2	[3, 19]
Evaluation Urgency	Urgent need to "establish an effective, objective, and complete evaluation system" for code-generation LLMs. [3]	2	[3, 32]
Job Automation	"Many users fear that AI will eventually replace human developers [...]" [19]	1	[19]

Table 4.9: Developers / Practitioners Opinions on AI and LLMs in Software Development

Opinion Category	Description	#	References
Productivity and Efficiency Gains	"The knowledge and reasoning capabilities of LLMs have the potential to result in significant efficiency gains." [13]	6	[2, 13, 19, 23, 24, 35]
Reliability Issues / Hallucinations	"LLMs sometimes produce highly convincing content that is factually wrong [...] the 'hallucination phenomenon'." [18]	6	[13, 18, 19, 22, 26, 32]
Output Variability and Trust Decline	"I get a different code each time [...] I lose a bit of confidence in the work." [13]	5	[13, 19, 32, 34, 35]
Quality of Generated Code	"Several quality issues: undefined variables, insecure comments, and code requiring significant revision." [26]	4	[4, 26, 27, 35]

Continued on next page

Augmentation of Human Capabilities	"LLMs are not merely a product of overhyped marketing; they represent a profound shift in how software is engineered. They should be seen as powerful tools that augment human capabilities rather than replace them." [24]	3	[19, 24, 35]
Learning and Skill Improvement	LLMs "[...] directly support learning processes through interaction and conceptual clarification." [2]	3	[2, 16, 24]
Knowledge Acquisition / Search Replacement	"LLMs present data in a concise and contextually relevant manner." [13]	2	[2, 13]
Controllability	Participants praised its "controllability [...] allowing modification and feature adjustment after generation." [4]	2	[4, 19]
Risk of Over-reliance and Cognitive Decline	"There is almost the danger that you stop thinking for yourself [...] people will tend to become lazy." [13]	2	[13, 19]
Investment of time with prompts	"You spend ages trying to cut the prompt so that it comes out right [...]" [13]	2	[13, 19]

Table 4.10: Organizations / Management Opinions on AI and LLMs in Software Development

Opinion Category	**Description**	**#**	**References**
Need for Human Oversight	LLM outputs "still necessitate extensive human review before implementation." [29]	7	[4, 17, 19, 29, 30, 33, 35]
Data Security, Privacy, and Corporate Bans	"Due to concerns regarding data privacy, when considering real-world practice, most software organizations tend to avoid using commercial LLMs and would prefer to adopt open source ones with training or fine-tuning using organization-specific data." [37]	6	[13, 15, 19, 35, 37, 39]
Operational Costs and Resource Constraints	Organizations must "consider the current limitations in terms of computational power or pay close attention to energy consumption," leading them to "tend to fine-tune medium-sized models". [37]	3	[13, 15, 37]
Strategic Goal for Developer Satisfaction	"We have to be able to compete on efficiency and job satisfaction [...] It is difficult to find software developers." [27]	2	[24, 27]

Continued on next page

Expectation Management Requirement	"SDOs must carefully manage their expectations towards AI tools." [27]	2	[19, 27]
Strategic Competitiveness	The "early adoption of LLMs in software engineering is crucial to stay competitive in this rapidly evolving landscape." [24]	1	[24]

The analysis of stakeholder opinions regarding the use of AI in Software Engineering, as synthesized in Tables 4.8 to 4.10, delineates a big picture of the impact that AI and LLMs, specially with the uprising in its usage, has and had on Developers and other stakeholders inside the Software Development Lifecycle and Engineering landscape. When collecting the opinions of stakeholders, it became clear that the different opinions should be separated by by the Stakeholder group and organized in different tables.

Starting from the first table, a fundamental tension exists between the immense capabilities of LLMs and their inherent fallibility, leading to skepticism among researchers and the public. This lack of certainty is reflected in the "Public/Researchers" group, where the General Trust/Perception, referenced in 6 articles, is that AI is a mixture of "software and sorcery" tying directly with the primary technical barrier is the Misinformation Risk, coming right after the previous opinion, meaning the generated code may execute without error, but fail to meet the intended task requirements. This risk is amplified because LLMs frequently exhibit high variability, where sending the same prompt multiple times yields different code each time. This unreliability directly mandates the single most prevalent opinion across all stakeholder groups (with 7 referenced articles): the Need for Human Oversight within the Organizations/Management opinions. This requirement is crucial because errors in LLM output, such as inconsistencies in design diagrams or incomplete implementations, can severely impact project integrity. Beyond technical quality, ethical concerns already exist and also arise, exemplified by the findings in the table.

In "Developers/Practitioners", while developers are eager to leverage LLMs for productivity and efficiency gains, which is the most-cited positive opinion for this group with 6 articles, there are significant challenges related to the practical integration of AI tools and the preservation of core technical competence. LLMs are also valued for knowledge acquisition, stated in 2 articles, as they allow developers to quickly bypass extensive search activities or dependence on colleagues [13]. However, this convenience introduces a documented danger of over-reliance and cognitive decline. Furthermore, maximizing the effectiveness of LLMs is not a seamless process but introduces a significant investment of time with prompts, referenced twice in the literature [13]. This necessity for highly specific prompt design means that using the tool effectively requires considerable skill and can reduce the efficiency gains it promises.

The "Organizations/Management" stakeholder group, the last group covered within the articles view LLM adoption as both a strategic necessity and a major risk management exercise focused heavily on controlling proprietary data and talent retention. The strategic decision to integrate LLMs is often tied

to Improving Job Satisfaction and organizational competitiveness [27]. By automating mundane tasks, AI helps retain valuable talent. However, this is heavily constrained by the group's most-cited concern: Data Security and Privacy, mentioned in 6 articles. The consensus strategy is to avoid using commercial LLMs and adopt open source ones with training or fine-tuning using organization-specific data to protect confidential and sensitive information. This ties into Operational Costs, where organizations when incorporating AI and LLMs in the workplace tend to fine-tune medium-sized models to manage computational limits. The novelty of the technology also requires a focus on the not so talked about expectation management, which is mentioned less but is equally as important, since Software Development Organizations have to be aware of the hype behind these tools and carefully manage their expectations when integrating AI tools in the Software Development Lifecycle landscape.

4.3.5 Industry sectors adopting AI and LLMs for Software Applications Development

This section will aim to answer the fifth research question by identifying which industry sectors are actively adopting AI for application development. It will further analyze the specific, sector-level benefits they report from this integration, grounding the research in real-world application.

Table 4.11: Industry sectors adopting AI and LLMs

Industry Sector	Description	#	References
Software Engineering, Software Development & Consulting Services	Software Development and Consulting is mentioned a lot, since they are the main sectors where AI and LLMs are being utilized in many different ways, such as AIOps, Cloud Management, Requirements Engineering, Software Testing, Quality Assurance, Code Generation and Development, Cybersecurity, Vulnerability Management, Refactoring, Code Optimization, Specialized Hardware, Scientific Computing and Emerging Technologies (like Blockchain/Crypto).	25	[2–4, 13–20, 22–24, 26–29, 32, 33, 35–39]
Finance, Banking, and Regulated Industries (Including Insurance)	Specialized LLMs can understand the "regulatory requirements, security needs, and performance constraints" especially in blockchain, cryptocurrency, smart contracts, and regulated environments. [24, 27, 29]	8	[4, 13, 17, 19, 24, 27, 29, 34]
Transportation and Safety-Critical Domains	The lack of transparency of LLMs is problematic, particularly in "safety-critical applications like healthcare, finance, or aerospace, where understanding the reasoning behind code is essential for ensuring security and correctness". [3]	7	[3, 15, 19, 24, 34, 37, 39]

Continued on next page

Industry Sector	Description	#	References
Healthcare, Medical, and Biomedical Research	AI "accelerates the coding process by converting natural language or abstract research intent (a vibe) into functioning software modules, dramatically shortening development cycles in biomedical research and clinical application." [1].	5	[1,4,5,18, 19]
Education and Training	LLMs "directly support learning processes through interaction and conceptual clarification". Students should be taught "how to specify AI, including functional, non- functional, and 'happy and unhappy path scenarios'" and learn "prompt engineering workarounds" and to "research and fact- check AI responses". [34]	5	[2,4,5,15, 34]
Legal Reasoning and Regulatory Compliance	LLMs demonstrate success across multiple fields, such as education, healthcare, and legal reasoning. Specifically, prompt engineering is used to "tailor LLMs for the US legal system, enabling the model to identify relevant cases and predict judicial outcomes with greater precision". [26]	4	[17,18,26, 34]
Digital Ecosystems (E-Commerce and Services)	Case studies involve the development of systems such as a food order and delivery system (FODS) and a smart wallet system (SWS). LLMs are applied in contexts like Online Retailers and the service industry. [4,29]	3	[4,17,29]

The analysis of industry sectors adopting AI and LLMs, detailed in Table 4.11, reveals a predictable but overwhelming concentration of AI adoption within the Software Engineering, Software Development, and IT Consulting industries. Given that these domains represent both the origin and primary proving ground for LLM-based tools, the literature emphasizes that the core use cases revolve around assisting with the actual coding process, particularly through code generation, completion, explanation, and refactoring. These capabilities are increasingly being integrated across the entire SDLC landscape. As a result, several sources highlight a gradual shift in the developer role, from writing code manually to orchestrating AI-enabled development workflows, where the human directs, validates, and refines the output of automated tools instead of performing every step manually. In consulting settings specifically, LLMs are described as valuable for helping teams rapidly familiarize themselves with client-specific business knowledge, accelerating onboarding and context transfer across projects. [4,27,29] Adoption of AI and LLMs also appears in specialized technical domains, which often involve complex, infrastructure-level, or mathematically intensive systems in which the LLMs must be adapted or augmented with structured knowledge to produce reliable outputs. [3,5,17,29,37]

Beyond the core IT sector, significant adoption is noted in high-stakes, regulated environments where

risks are severe. This group includes Finance, Banking, and Regulated Industries, referenced in 8 articles of the literature, and Transportation and other Safety-Critical Domains, referenced in 7 articles, Healthcare and Biomedical Research, mentioned in 5 articles and last but not least Legal Reasoning with 4 articles. In these domains, LLMs are introduced more cautiously, with emphasis on reliability, auditability, and compliance with regulatory standards. However, the literature consistently stresses and makes an effort to make a non-negotiable that in all such high-stakes environments, extensive human review and verification remain mandatory, particularly where incorrect or unverifiable outputs may result in safety risks, compliance failures, or legal liability [1, 3, 26, 29, 33].

The Education and Training sector, referenced in 5 articles, is emerging as a notable area of adoption, particularly in contexts related to software engineering education and professional skill development. Studies highlight that LLMs increasingly function as instructional support tools, helping learners understand programming concepts, debug code, and explore alternative solution strategies through interactive dialogue-based guidance. Rather than replacing educators, LLMs act as adaptive learning companions, offering step-wise reasoning and tailored explanations that align with the learner's current level of mastery. The literature further reports their use in automated teaching pipelines, supporting tasks such as generation of programming exercises, personalized feedback, and formative assessment. This integration also aligns with a broader pedagogical shift: as routine coding tasks become more easily automated, software engineering curricula increasingly emphasize conceptual understanding, architectural decision-making, and critical evaluation, while LLMs provide practice-oriented support for iterative learning [2, 4, 5, 15, 34].

4.4 Discussion

This section discusses the relevant information extracted from the articles selected for the SLR, synthesizing the findings from the five research questions to construct an integrated and broad view of the current landscape of AI and LLM adoption in software development. The results confirm the innovative and transformative potential of these technologies, but also highlight significant risks without any solid and clear use of strict methodologies, that shape their use across industry and professional practice.

The findings for the first research question show that AI and LLMs offer relevant benefits and enhancements to software development overall. The two most widely reported benefits in Table 4.5 indicate that AI tools can meaningfully accelerate development workflows and elevate output standards. Importantly, these advantages extend across the entire SDLC appearing not only in coding, but also in requirements and early design phases of the development overall, as well as in automated testing, debugging, and repair. This demonstrates that AI is not merely a code-generation tool, but a broad co-creative and analytical partner within software engineering practice.

With regard to the consequences of AI and LLMs usage in Software Development, the results of the second research question reveals that these benefits that were previously discovered are also inseparable from substantial and important risks. The three most frequently identified drawbacks within the articles in Table 4.6 directly undermine the foundational benefit of improved code quality. This introduces a central tension in current software engineering practice with AI and its mission, it promises acceleration and enhancement, but its integration and further use can generate new points of failure that are costly, unpredictable, and often difficult to detect. Legal, ethical, and intellectual property uncertainties are also connected to the previous risks since they further complicate adoption and incorporation. Moreover, the lack of standardized benchmarks and evaluation metrics makes it challenging to objectively assess the genuine contribution of AI tools to quality assurance and security within the SDLC, hindering accurate comparison and validation of their effectiveness.

The findings for the next research question which covers the Methodologies and Management Methods found within the literature show that current industry practice is not built around full automation at all, but around control. The most prevalent methods documented in Table 4.7 like In-Context Learning, Prompt Engineering and Chain-of-Thought Prompting all aim to shape and constrain AI behavior before code is generated. Meanwhile, the Human-In-The-Loop paradigm makes sure that developers remain directly responsible for reviewing, validating, and approving all outputs maintaining the human presence throughout the whole development process, ensuring a quality experience. Rather than replacing developers, AI currently functions as a tool that must be guided and supervised. The presence of multiple emerging frameworks, including Agile-based human-AI collaboration models mentioned in the articles, indicates that the field is actively searching for structured and repeatable integration patterns and methodologies, but no universally accepted approach has yet emerged.

This tension is reflected clearly in the fourth research question, which results are documented in Tables 4.8 to 4.10. Developers and practitioners express enthusiasm about productivity and efficiency gains, but are equally vocal in concerns regarding reliability issues, hallucinations and output variability with AI usage and outputs. Organizations and management echo this trade-off because while aiming to improve developer satisfaction and competitiveness, they prioritize human oversight during AI-assisted development, data security and privacy protection, often restricting or prohibiting the use of commercial LLMs for confidentiality reasons. Both groups converge on the consensus that clear expectations must be established and that AI systems should augment developer expertise while maintaining a controlled level of trust regarding its capabilities.

Finally, the last research question results reveal that adoption is not uniform. The sectors most actively integrating AI is Software Engineering, Software Development and Consulting, where experimentation and rapid iteration are strategically valuable and lower-risk and LLMs are used both as operational tools and as accelerators of knowledge transfer. In contrast, sectors characterized by regulatory sensi-

tivity or safety-critical operations such as Finance, Banking, Transportation, Aerospace, and Healthcare, all documented in Table 4.11, exhibit cautious and controlled adoption, often requiring stronger guarantees of interpretability, security, and auditability before deployment. This reinforces the conclusion that the maturity of LLM integration is influenced not only by technological capability, but also by the risk tolerance and domain-specific requirements of the adopting organization.

To address the need for a clearer mapping between AI capabilities and the Software Development Lifecycle, Table 4.12 synthesizes the findings of the SLR by outlining the primary impacts, benefits, and challenges of AI and LLM usage across individual SDLC phases.

Table 4.12: Impact of AI and LLMs across Software Development Lifecycle Phases

SDLC Phase	Main AI Contributions	Main Challenges
Requirements Analysis	Support for requirements elicitation, clarification, summarization, and stakeholder communication; rapid exploration of solution ideas through conversational interfaces.	Risk of ambiguity amplification, overconfidence in AI-generated requirements, and lack of domain context; increased need for human validation early in the process.
Design	Generation of architectural alternatives, design patterns, API suggestions, and exploratory prototypes.	Limited understanding of system-wide constraints; risk of inconsistent or suboptimal architectural decisions without expert oversight.
Implementation	Code generation, refactoring, boilerplate reduction, and rapid prototyping; strong association with practices such as Vibe Coding.	Hallucinated code, inconsistent outputs, security vulnerabilities, and reduced trust due to output variability; heavy reliance on human review.
Testing	Automated test case generation, test data creation, bug localization, and support for exploratory testing.	Difficulty validating correctness of AI-generated tests; increased testing complexity due to unpredictable AI-generated code.
Deployment	Support for configuration scripts, infrastructure-as-code templates, and documentation generation.	Risk of misconfiguration or insecure defaults; limited contextual awareness of operational environments.
Maintenance	Assistance with debugging, documentation updates, code comprehension, and technical debt identification.	Potential over-reliance on AI explanations; challenges in maintaining long-term system understanding and accountability.

Overall, the SLR provides a comprehensive understanding of the current state of AI-driven software development. The findings demonstrate that while AI and LLMs offer meaningful benefits, the absence

of a robust, standardized, and secure framework and/or methodologies for integrating these tools into the Software Development Lifecycle remains the largest barrier to widespread adoption. The promise and current new "Vibe Coding" paradigm is clear, trying to make software development as high-level conceptual expressions rather than manual syntax construction, but realizing this vision requires overcoming significant challenges in reliability, safety, organizational governance and much more. Thus, the future of AI in software development depends not on improving the raw capability of models alone, but on creating methodologically sound, transparent, and trust-centered processes and workflows for using them effectively.

5

Research Problem

This chapter presents the first activity of the DSRM: the identification of the research problem and its motivation. The definition of objectives, the second activity, is presented together with the design of the artefact that satisfies them in Chapter 6, since the objectives derived here are what that chapter's design is answerable to.

As shown in the results reported in Chapter 4, a lot of sectors can take advantage of the AI and LLMs transformative assets, incorporate it into their workflow. Although AI has been in the Software Development landscape for a while now, due to the highly dynamic and fast-evolving nature of the field, new models and new tools are being created nearly every month, and AI usage in general has never been so prevalent not only in the Software Development field but in other industry sectors as well.

The conclusions drawn from the information collected in the systematic literature review, indicate that while many benefits can be attained from the use of AI, many challenges and disadvantages are in the way of its successful implementation and usage. Therefore the real main challenge organizations face and one of the most important consequences that is mentioned within the literature is the lack of established, solid and clear methodologies, that shape the proper use of AI and LLMs within the Software Development realm and across the industry and professional practice. [13–16]

Although several frameworks and approaches that incorporate AI into the development process have been proposed, as summarised in Table 4.7, these solutions differ significantly in scope, assumptions,

and level of abstraction. They often lack a shared foundation regarding lifecycle integration, management practices, and evaluation criteria. Consequently, organizations face difficulties not only in adopting AI-supported development practices, but also in objectively assessing their impact on software quality, security, and maintainability due to the lack of standardized benchmarks and performance metrics; ultimately proving that the field is actively searching for structured and repeatable integration patterns and methodologies. [3]

This problem is further amplified by the fact that AI increasingly influences all phases of the SDLC. Due to this problem, companies are seeking for a new methodological evolution to fully embrace, utilize, incorporate and implement AI and LLMs into their workflow. Traditional methodologies such as Waterfall, Scrum, Kanban, and the Agile Manifesto introduced substantial improvements in adaptability and collaboration, but they were not designed for development environments in which AI or LLMs actively contribute to software creation. [4]

As a result and as AI-assisted development becomes more prevalent, modern development teams experience new dynamics, including the rise of Vibe Coding, accelerated implementation, emerging bottlenecks created by AI in earlier phases such as requirements clarification and later phases such as testing, and a stronger emphasis on delivering product value rather than merely increasing feature output. Together, these changes indicate that existing methodologies may no longer fully align with emerging AI-augmented development dynamics. Existing traditional frameworks often fall short in AI-driven contexts because:

- high-speed AI-assisted coding accelerates implementation but creates pressure upstream (requirements clarification) and downstream (testing, verification, and maintenance);
- they lack explicit structures for AI governance, prompt engineering, model reliability assessment, and risk mitigation;
- they assume all contributors are humans, whereas AI models introduce a new category of collaborator.

Two independently reported sets of figures make this concrete, and they point in the same direction: the limiting factor is the process surrounding the tool rather than the capability of the model itself.

The first concerns correctness against security. Veracode's 2025 assessment of over one hundred large language models across Java, Python, C# and JavaScript found that 45% of generated code samples introduced an OWASP Top 10 vulnerability, rising to 72% for Java, and that models failed to defend against cross-site scripting in 86% of the relevant samples. The report's central observation is not the failure rate itself but its independence from model capability: newer and larger models produced functionally and syntactically correct code more often, while security performance remained flat [9].

Capability alone therefore does not converge on secure output, which is what makes explicit validation a structural requirement rather than a matter of discipline.

The second concerns where the work accumulates. Faros AI's 2025 analysis of engineering telemetry from more than ten thousand developers across 1,255 teams found that high AI adoption coincided with 21% more tasks completed and 98% more pull requests merged, but also with a 91% increase in pull request review time, a 154% increase in average pull request size, and a 9% increase in defects per developer, with no significant correlation between AI adoption and improvement at the organizational level [12]. Acceleration at the point of authorship does not disappear; it relocates to human review, and it does not by itself reach delivery outcomes.

Both sources are industry-reported rather than peer-reviewed, and are cited here as such. Their value lies in agreeing on the mechanism, not in the precision of any single figure: neither security nor delivery improves as a by-product of faster generation, and both depend on where validation and review are positioned in the process.

Therefore, the core research problem addressed in this work is the lack of patterned guidance, strict methodologies and standardized evaluation metrics that can be defined through a structured, process-oriented framework that supports the systematic integration and management of AI and LLM-based tools across the Software Development Lifecycle, while preserving human oversight, software quality and security, and organizational accountability. This gap limits organizations' ability to fully and sustainably realize the potential benefits of AI-assisted software development.

The objectives that a solution to this problem must satisfy, and the design that answers them, are presented in Chapter 6.

6

Research Proposal

Contents

This chapter presents the second and third activities of the DSRM: the definition of objectives for a solution to the research problem formalised in Chapter 5, and the design of the artefact that satisfies them, a process-oriented framework for governed Human-AI collaboration across the SDLC. The framework is described here as a methodology, independently of any implementation; its instantiation as a working system is the subject of Chapter 7.

6.1 Objectives

From the systematic literature review conducted in Chapter 4, it was possible to gather an overview of the current status of the topic of AI and LLMs usage adoption in regards to Software Development. With the analysis of the consequences and benefits that these tools have, a research problem was defined in Chapter 5, namely, the lack of patterned guidance, strict methodologies and standardized evaluation metrics to fully make the most of out of the potential that AI and LLMs can bring to an information rich environment. Therefore, the research objectives of this work are defined to directly address that research problem. This problem, grounded in the findings of the Systematic Literature Review, concerns the lack of structured guidance, methodological rigor, and standardized evaluation practices for integrating AI and LLMs into software development processes. Table 6.1 summarises how the research objectives are formulated to resolve this problem, linking the identified gaps to concrete framework objectives.

Table 6.1: Mapping between Research Questions, Key Findings, Problem Aspects, and Framework Objectives

RQ	Key Finding from SLR	Problem Aspect Addressed	Resulting Framework Objective
RQ1	The analysis of benefits and usage patterns shows that AI and LLM usage is unevenly distributed across SDLC phases and lacks structured integration.	Lack of structured guidance on where and how AI should be applied across the SDLC	Define a process framework that explicitly maps AI-supported activities to specific SDLC phases, clarifying where and how AI tools should be applied.
RQ2	The analysis of consequences and limitations highlights recurring risks related to reliability, validation, security, and insufficient human oversight.	Insufficient methodological support for managing risks, quality, and accountability in AI-assisted development	Integrate governance mechanisms, human-in-the-loop practices, and quality assurance checkpoints into the framework.
RQ3	The review of existing methodologies reveals significant variation among approaches and a lack of consensus on how to manage AI adoption in software development.	Fragmentation and lack of consensus in existing AI adoption methodologies	Consolidate best practices into a unified, process-oriented framework that provides repeatable guidance for managing human-AI collaboration.

Together, these objectives operationalize a solution to the identified research problem by translating observed gaps in current practice into concrete design goals for the proposed framework. By explicitly mapping AI usage to SDLC phases, introducing governance and human-in-the-loop mechanisms, and

consolidating fragmented practices into a unified process model, the framework aims to resolve the lack of structured guidance and methodological clarity currently limiting the effective and responsible adoption of AI and LLMs in software development. The remainder of this chapter presents the framework designed to meet these objectives.

6.2 Purpose and Scope

Scope in: AI-assisted activities across every SDLC phase; practices for human-AI collaboration, prompt formulation and model selection; human-AI interaction patterns; governance metrics and oversight mechanisms. Scope out: training or fine-tuning of models; model architecture design; infrastructure-level MLOps pipelines; tool-specific implementation detail.

6.3 Design Principles

Eight principles, each with its grounding: Human-in-the-Loop by Design (grounded in the adoption/trust gap and in security benchmarking of agent-generated code); AI as Augmentation, Not Substitution; Process-Level Governance of AI Usage; SDLC-Wide and Phase-Aware Integration; Hats, Not People; Explicit Responsibility and Accountability; Risk-Proportional Validation and Control; Spec-Anchored Continuity (grounded in spec-driven development as an emerging industry practice, cited with its own stated limitation).

6.4 Lifecycle Phases and the AI Interaction Mapping

Present the phase model end to end: Strategic Alignment; Discovery & Requirements; Design, Prototyping & Architecture; Implementation; Testing; Deployment; Maintenance. For each: what enters, what leaves, what the human is accountable for, and which control is mandatory. State explicitly the deliberate divergence from AWS's AI-Driven Development Lifecycle (AI-DLC) [42] (a decoupled Testing phase and a distinct Maintenance phase with its own diagnostic firewall) and justify it. Note that AI-DLC's publicly reported productivity figures, of the order of ten to fifteen times, are vendor-reported and not independently verified, and must be presented as such if quoted. Emphasise that Maintenance loops back into a named phase rather than patching in place - this is the direct answer to any reading of the model as linear or Waterfall.

6.5 Roles and Responsibilities

Define each hat as a function that must be performed rather than a person who must exist: Project Manager; the Trio (Product Owner, Tech Lead, Design Lead); Bolt Master; Developer; QA. Justify the removal of the DevOps Alliance role. Cite the Spotify Model alongside its own retraction, and Team Topologies as the more rigorous structural vocabulary. Make clear that on a solo or small team the gates still apply, self-checked rather than checked by a second person.

6.6 Governance Mechanisms and Quality Gates

6.6.1 Bolts as the Unit of Execution

The micro-cycle: task breakdown and architectural lock, context alignment and assignment, trigger, Consistency Factor generation, generation and refinement, validation and story completion, and the Fix-Bolt exception.

6.6.2 Spec-Anchored Continuity

The persistent context artefact, split into a global layer (immutable architectural rules, tech-stack decisions, security policies) and a local layer (the approved functional specification and the formal technical specification). Explain why narrowing the model's working context is a control, not a convenience, building on the spec-driven development precedent introduced in Section 3.5.

6.6.3 Quality Gates

Discovery, Design, Implementation, Testing and Deployment gates: who signs off, against what evidence, and what happens on failure.

6.6.4 Governance Metrics

Bolt Cycle Time, Context Traceability Rate, AI Defect Escape Rate. Map these onto the NIST AI RMF functions and ISO/IEC 42001 vocabulary as shared language, explicitly not as a compliance claim.

6.7 Work Organisation

Hierarchy of work (Unit of Work, User Stories, Tasks); the Bolt Backlog as a continuous prioritised queue; the Bolt Board. Ground the departure from sprint cadence in the RCT and telemetry evidence introduced in Section 1.1: more change volume, longer review, flat delivery.

6.8 Operational Playbooks

Mob Elaboration (requirements); Design & Prototyping; Maintenance & Evolution; Fix-Bolt; QA Validation; Deployment & Release.

6.9 Positioning Against Alternatives

Engage each alternative honestly, with its documented failure mode: (i) no framework at all; (ii) AI-DLC as published and unmodified [42]; (iii) keeping Scrum or SAFe cadence and treating AI as a faster IDE; (iv) metrics-only governance with no phase structure. State plainly what each does better than this framework, and where the evidence says it falls short.

6.10 Summary

Close by restating how the framework answers DRQ1 and DRQ2, and hand over to Chapter 7, which demonstrates that these constructs are implementable rather than merely describable.

7

Apex: The Reference Implementation

Contents

The framework presented in Chapter 6 describes a methodology. This chapter presents Apex, the working software system that instantiates it, and which serves both as evidence that the framework's governance constructs are implementable and as the vehicle through which the framework is demonstrated and evaluated in Chapters 8 and 9. In DSRM terms, the framework and Apex are two artefacts produced by the same Design and Development activity: a method and its instantiation.

This chapter also documents the design iterations that shaped both artefacts. Practitioner feedback gathered during construction did not merely refine the tool; it changed the framework itself. Recording that path is what makes the DSRM iteration explicit rather than implied.

7.1 Purpose and Positioning

State the positioning directly: a coding agent accelerates the writing of code, whereas Apex governs whether that code still answers the originating question. Every artefact carries a stable specification identifier back to its requirement, so drift is surfaced rather than silent. Note also what Apex deliberately does not do: no model training or fine-tuning, no infrastructure provisioning beyond generating scripts for human or CI execution, and no replacement of the project management tool it sits above.

7.2 Architecture

Split-stack overview: Next.js frontend, Python/FastAPI backend, and the separation between them. Spec-anchored storage, and how project isolation is enforced. The concurrency model and why it is a deliberate single-writer design. The proxy-first approach to external integrations and the security rationale behind it. The provider abstraction that keeps the system independent of any single model vendor.

Include the architecture figure here, with a caption that explains what the reader should take from it rather than restating the components.

7.3 Operationalising the Framework

Build a mapping table: framework construct (left) against the implemented mechanism (right), covering each lifecycle phase, each quality gate, Spec-Anchored Continuity as versioned specification files, stable specification identifiers, the phase-status flow, Bolt tracking and its cycle-time metric, spec-drift detection and feedback routing, and the traceability graph. Every row must point at something that exists, not something planned.

Include the user-flow figure here, showing how a practitioner actually walks the phases.

7.4 Design History and Practitioner Feedback

Present the evolution as a sequence of design decisions, each with: the feedback or observation that prompted it, the change made to the framework and/or the tool, and the rationale. Report changes that were considered and rejected, and changes that were made and later reverted, with the reason. Negative results here strengthen the evaluation rather than weakening it.

Record explicitly which framework-level changes originated in this feedback rather than in the tool alone - for example the removal of a role from the framework, and the demotion of a mandatory artefact to optional. These are the clearest evidence that evaluation fed back into design.

Include a table summarising the interview rounds: date, participant profile (role, seniority, sector), format, and the design changes attributable to each round. Anonymise participants; the interview instrument itself belongs in an appendix.

7.5 Quality Assurance of the Implementation

Summarise the testing strategy across backend, frontend unit, and end-to-end layers, the CI gating structure, and the deployment pipeline. Report current suite sizes and what each layer is responsible for catching.

Include a short, honest account of defects found and fixed during construction, particularly those with methodological significance - for example failures of project isolation, or cases where generated output required server-side reconciliation before it could be trusted. These are evidence for the framework's own premise that AI output requires validation.

7.6 Limitations of the Implementation

Separate clearly the limitations of Apex as a system from the limitations of the framework as a methodology. Where a metric is implemented as a proxy rather than the real measurement, say so explicitly and name the substitution.

8

Demonstration

Contents

This chapter reports the Demonstration activity of the DSRM: the application of the framework, through its reference implementation, to a real software development project. The purpose of the demonstration is to show that the artefact can be used to address the research problem in a genuine setting, and to generate the evidence on which the evaluation in Chapter 9 rests.

8.1 Demonstration Design

State the unit of analysis, the duration, the number of iterations, and which phases of the framework are exercised. Define in advance what will be recorded at each decision point: which artefacts, which measurements, and which observations.

State the researcher's own role explicitly. Where the author is also a participant, name that as a threat to validity here rather than leaving it to the evaluation chapter.

8.2 Context

Describe the organisational setting, the team composition and the hats each participant wears, the existing tooling and process baseline, and the nature of the software being built. Establish the baseline concretely enough that a reader can judge what the framework changed.

Characterise the problems present in the setting before the framework was applied, mapping them back to the gaps identified in Chapter 4: inconsistent validation of generated code, upstream pressure on requirements as prototyping accelerates, testing complexity, and unclear responsibility boundaries over generated artefacts.

8.3 Application of the Framework

Walk each phase in turn. For each: what the AI produced, what the human accepted, modified or rejected, which gate was applied, and what the gate decided. Include real artefacts - generated scenarios, specifications, task breakdowns, test plans - as figures or listings rather than describing them.

Record the loop-backs explicitly. A demonstration in which nothing was rejected and nothing looped back is not evidence that the framework works; it is evidence that it was not exercised.

8.4 Observations

Report what was observed at the decision points defined in Section 8.1, including the governance metrics the framework defines. Report observations that were unfavourable to the artefact with the same prominence as those that were favourable.

Where the framework proved awkward, over-specified, or was silently bypassed in practice, record it. These observations are the most valuable input to the refinement discussion in Chapter 10.

8.5 Summary

Summarise what the demonstration establishes and, equally importantly, what it does not. Hand over to Chapter 9.

9

Evaluation

Contents

This chapter reports the Evaluation activity of the DSRM. Following Pries-Heje et al.'s distinction between when and how an artefact is evaluated [25], the evaluation carried out here is ex post, since it assesses the artefact after construction and on the basis of its actual use, and naturalistic, since it takes place in a real organisational setting with real users rather than in a laboratory.

9.1 Evaluation Strategy

Two artefacts were produced in Chapters 6 and 7: a framework and its instantiation. They are not evaluated by the same means, and conflating them would weaken both claims. Table 9.1 therefore

states which instrument evaluates which artefact, and which research question it addresses.

Table 9.1: Evaluation instruments, target artefact, and research question addressed

Instrument	Artefact evaluated	DRQ	What it establishes
Demonstration in a real project	Framework and instantiation	DRQ3, DRQ4	That the artefact is applicable and produces the intended artefacts and decisions
System Usability Scale (SUS)	Instantiation (Apex)	DRQ4	Perceived usability of the supporting tool
NASA Task Load Index (NASA-TLX)	Instantiation (Apex)	DRQ4	Perceived cognitive workload imposed on the practitioner
Practitioner interviews	Framework	DRQ1, DRQ2	Practicality, feasibility, and perceived value in real workflows
Agile and Scrum expert interviews	Framework	DRQ1, DRQ2	Alignment with, and departure from, established delivery methodology
Analytical assessment against criteria	Framework	DRQ1, DRQ2	Degree to which stated design criteria are satisfied

State plainly that SUS and NASA-TLX measure the instantiation, not the methodology, and that the framework's own validity rests on the interviews, the analytical assessment, and the demonstration evidence. Positioning this limitation as a deliberate design of the evaluation is far stronger than conceding it under questioning.

9.2 Participants

Report the number of participants, their roles, seniority, sector, and prior exposure to AI-assisted development. Note that SUS is generally considered to yield stable results from around twelve respondents, and state honestly what the achieved sample supports and what it does not.

Describe recruitment, informed consent, anonymisation, and data handling.

9.3 Instruments and Procedure

9.3.1 System Usability Scale

The SUS [43] is a ten-item instrument scored on a five-point scale of strength of agreement, administered after the participant has completed a defined set of tasks. Participants responding in Portuguese are

given the validated European Portuguese version [44], with the only adaptation being the substitution of the system name for the generic referent used in the published items; participants responding in English are given the original wording. The two language versions are reported separately rather than pooled.

Report the scoring procedure and state the interpretation scale used, including the adjective ratings of Bangor et al. [45] and the curved grading scale of Sauro and Lewis [46], so that a single number is not left to speak for itself. Report the Usability and Learnability sub-scales separately [47]. State plainly that the score is not a percentage. State also the limits the Portuguese validation itself reports: construct validity was established, but inter-rater reliability was weak (ICC = 0.36) and agreement was 76.67 per cent, and the validation sample was drawn from the general community rather than from software practitioners.

9.3.2 NASA Task Load Index

The NASA-TLX [48] measures perceived workload across six subscales: mental demand, physical demand, temporal demand, performance, effort and frustration. This evaluation uses the unweighted Raw NASA-TLX variant [49], omitting the fifteen pairwise weighting comparisons. The instrument is administered once after each of the marked tasks rather than once at the end of the session, which is the instrument's intended per-task use and yields a workload profile per lifecycle phase rather than a single aggregate.

Justify the Raw variant explicitly: under per-task administration the weighting procedure would require fifteen comparisons for each of six tasks, which is not feasible within the session, and there is no second condition against which weights would earn their cost. Report the response scale actually used, and if it departs from the original twenty-interval line, state the departure and that the resulting values are not directly comparable to studies using the original scale. Note the trade-off that repeated administration invites anchoring and end-of-session fatigue.

Define the task set both instruments are administered against, and keep it identical across participants. Place the full instruments in an appendix and reference them here.

9.3.3 Semi-Structured Interviews

Describe the interview protocol, the two participant groups (practitioners and Agile experts), the duration, and the analysis method used to derive themes from the transcripts. Place the interview guide in an appendix.

9.3.4 Analytical Assessment

State the success criteria in advance - clarity, completeness, alignment with SDLC phases, support for human-in-the-loop practice, and governance of AI-enabled activity - and the three-level ordinal scale used to assess each. Each rating must carry a short qualitative justification grounded in observed usage, artefacts, or interview evidence rather than in the author's judgement alone.

9.4 Results

Report SUS results: per-participant scores, mean, and dispersion. Given the expected sample size, report the distribution rather than the mean alone, and avoid inferential statistics the sample cannot support.

Report NASA-TLX results per subscale, not only the aggregate. The subscale profile is the informative part: a tool that raises mental demand while lowering frustration and effort tells a specific and reportable story about where the framework moves the practitioner's burden.

Report the interview findings as themes, with representative anonymised quotations. Report disconfirming evidence explicitly.

Report the analytical assessment as a table of criteria, ratings, and justifications.

9.5 Threats to Validity

Address construct, internal, external, and conclusion validity. Name specifically: the author's dual role as designer and participant; the small and non-random sample; the single organisational setting; the short observation window, which precludes any claim about long-term maintainability or organisational change; and the fact that the instruments measure perception rather than delivered software quality. For each threat, state the mitigation applied, or state plainly that none was possible.

9.6 Discussion

Synthesise across instruments. Where they agree, say so; where they disagree - for instance a favourable usability score alongside interview scepticism about the process overhead - treat the disagreement as a finding rather than smoothing it over, and read it against the threats to validity stated above.

9.6.1 Answering the Research Questions

Answer DRQ1, DRQ2, DRQ3 and DRQ4 from Section 1.3 in turn, each grounded explicitly in the evidence reported above (results, demonstration, interviews) rather than merely asserted. DRQ1 and DRQ2 concern the framework and are answered primarily from the practitioner and Agile-expert interviews and the analytical assessment; DRQ3 is answered from the construct-to-mechanism mapping in Section 7.3 read together with the implementation limitations, reporting which constructs were realised in full, which were realised as proxies, and which resisted implementation, since the existence of Apex alone establishes only that something was built; DRQ4 is answered from SUS, NASA-TLX, and the demonstration. This closes the loop the introduction opened: each research question is answered here with evidence, not merely restated in the conclusion.

10

Conclusion

Contents

10.1 Summary and Contributions

Restate the problem, the artefacts produced, and what the evaluation established. Summarise, rather than re-derive, how each research question from Section 1.3 was answered; the evidence-grounded answers themselves belong in Section 9.6.1. Keep this to what the evidence actually supports.

10.2 Communication

This section addresses the Communication activity of the DSRM.

Report the dissemination plan and its current status: the scientific article reporting the SLR and its target venue; the report presenting the framework; and this dissertation, formally presented and defended before an academic examination committee. Update the status of each submission before final delivery.

10.3 Limitations

State the limitations of the research honestly: the framework's core constructs are an original synthesis rather than being drawn wholesale from a single validated source; several grounding sources are recent, vendor-reported, or not yet peer-reviewed, and should be cited as such; the governance-standards vocabulary is used as shared language and not as a compliance claim; and the evaluation covers early adoption in a single setting, so it cannot speak to sustained organisational change.

10.4 Future Work

Derive future work from the evaluation findings rather than listing generic extensions. Candidates: application across multiple organisations and team sizes to test generalisation; longitudinal measurement of defect escape and maintainability, which the current evaluation window cannot reach; support for project management backends beyond the one currently integrated; and replacing proxy metrics with direct measurement where production telemetry becomes available.

Declaration of Generative AI and AI-Assisted Technologies in the Writing Process

During the preparation of this work, the author used Claude Code, with the Sonnet 5, Opus 5 and Fable 5 models, together with Google Gemini and Google NotebookLM, to improve the wording and clarity of the manuscript and to support the organisation and review of source material. After using these tools, the author reviewed and edited the text as needed and takes full responsibility for the content of the publication. The final manuscript reflects the author's own analysis, results, and conclusions.

This declaration concerns the use of these technologies in the writing process only. The use of generative AI as the object of study of this research, and within the artefacts it produces, is described in Chapter 6 and Chapter 7.

Bibliography

[1] J. H. Moore and N. Tatonetti, “Vibe coding: a new paradigm for biomedical software development,” *BioData Mining*, vol. 18, no. 1, pp. 1–3, 2025.

[2] A. Zamfiroiu, C.-B. Păştinică, C. Crăciun, C. Ciutacu, and T. Luca, “Exploring Conversational Programming through the CHOP Paradigm and Artificial Intelligence,” *Informatica Economica*, vol. 29, no. 2, pp. 5–17, 2025.

[3] I. Ahmed, A. Aleti, H. Cai, A. Chatzigeorgiou, P. He, X. Hu, M. Pezzè, D. Poshyvanyk, and X. Xia, “Artificial Intelligence for Software Engineering: The Journey So Far and the Road Ahead,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–27, 2025.

[4] S. Zhang, Z. Xing, R. Guo, F. Xu, L. Chen, Z. Zhang, X. Zhang, Z. Feng, and Z. Zhuang, “Empowering Agile-Based Generative Software Development through Human-AI Teamwork,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 6, pp. 1–46, 2025.

[5] W. Hou and Z. Ji, “Comparing Large Language Models and Human Programmers for Generating Programming Code,” *Advanced Science*, vol. 12, no. 8, pp. 1–12, 2025.

[6] Stack Overflow, “2025 Developer Survey: AI,” Annual practitioner survey, industry-reported, not peer-reviewed, 2025. [Online]. Available: https://survey.stackoverflow.co/2025/ai/

[7] DORA and Google Cloud, “State of AI-assisted Software Development 2025,” Industry research report, not peer-reviewed, 2025. [Online]. Available: https://dora.dev/dora-report-2025/

[8] S. Zhao, D. Wang, K. Zhang, J. Luo, Z. Li, and L. Li, “Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks,” arXiv:2512.03262, preprint, not yet peer-reviewed, 2025. [Online]. Available: https://arxiv.org/abs/2512.03262

[9] Veracode, “2025 GenAI Code Security Report: Assessing the Security of Using LLMs for Coding,” Industry report, not peer-reviewed. Assessment of over 100 large language models across Java, Python, C# and JavaScript, July 2025. [Online]. Available: https://www.veracode.com/wp-content/uploads/2025_GenAI_Code_Security_Report_Final.pdf

[10] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, “The Impact of AI on Developer Productivity: Evidence from GitHub Copilot,” arXiv:2302.06590, preprint, not published in a peer-reviewed venue, 2023. [Online]. Available: https://arxiv.org/abs/2302.06590

[11] J. Becker, N. Rush, E. Barnes, and D. Rein, “Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity,” arXiv:2507.09089, METR, preprint, not yet peer-reviewed, 2025. [Online]. Available: https://arxiv.org/abs/2507.09089

[12] Faros AI, “The AI Productivity Paradox Report 2025,” Industry telemetry report, not peer-reviewed. Sample of over 10,000 developers across 1,255 teams, July 2025. [Online]. Available: https://www.faros.ai/blog/ai-software-engineering

[13] L. Banh, F. Holldack, and G. Strobel, “Copiloting the future: How generative AI transforms Software Engineering,” *Information and Software Technology*, vol. 183, 2025.

[14] M. Farhanna, N. Paramitha, M. Susi, H. Surya, A. Yanti, P. Dea, and S. Deni, “Exploring the Use of Generative AI in Software Development: A Preliminary Study,” in *E3S Web of Conferences*, vol. 645, 2025, p. 04002.

[15] A. Hemmat, M. Sharbaf, S. Kolahdouz-Rahimi, K. Lano, and S. Y. Tehrani, “Research directions for using LLM in software requirement engineering: a systematic review,” *Frontiers in Computer Science*, vol. 7, 2025.

[16] C. Gao, X. Hu, S. Gao, X. Xia, and Z. Jin, “The Current Challenges of Software Engineering in the Era of Large Language Models,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–30, 2025.

[17] H. A. Al-Hashimi, R. A. Khan, H. S. Alwageed, A. M. Algarni, S. Ayouni, and A. O. Almagrabi, “Exploring the role of generative AI in enhancing cybersecurity in software development life cycle,” *Array*, vol. 28, p. 100509, 2025.

[18] X. Bai, S. Huang, C. Wei, and R. Wang, “Collaboration between intelligent agents and large language models: A novel approach for enhancing code generation capability,” *Expert Systems With Applications*, vol. 269, 2025.

[19] M. Basha and G. Rodríguez-Pérez, “Trust, transparency, and adoption in generative AI for software engineering: insights from twitter discourse,” *Information and Software Technology*, 2025.

[20] X. Chen, C. Gao, C. Chen, G. Zhang, and Y. Liu, “An Empirical Study on Challenges for LLM Application Developers,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 7, pp. 1–37, 2025.

[21] Y. Dong, X. Jiang, Z. Jin, and G. Li, “Self-Collaboration Code Generation via ChatGPT,” *ACM Transactions on Software Engineering & Methodology*, vol. 33, no. 7, pp. 1–38, 2024.

[22] U. K. Durrani, M. Akpinar, M. F. Adak, A. T. Kabakus, M. M. Ozturk, and M. Saleh, “A Decade of Progress: A Systematic Literature Review on the Integration of AI in Software Engineering Phases and Activities (2013-2023),” *IEEE Access*, vol. 12, pp. 171 185–171 204, 2024.

[23] Y. Han and C. Lyu, “Multi-stage guided code generation for Large Language Models,” *Engineering Applications of Artificial Intelligence*, vol. 139, 2025.

[24] M. A. Haque, “LLMs: A game-changer for software engineers?” *BenchCouncil Transactions on Benchmarks, Standards & Evaluations*, vol. 5, no. 1, pp. 30–41, 2025.

[25] J. R. V. Jan Pries-Heje, R. Baskerville, “Strategies for Design Science Research Evaluation,” *European Conference on Information Systems (ECIS)*, no. 1, pp. 1–13, 2008.

[26] S. K. Jabrw and Q. I. Sarhan, “A Systematic Survey on Large Language Models for Code Generation,” *ARO-The Scientific Journal of Koya University*, vol. 13, no. 2, 2025.

[27] V. V. Jensen, A. Alami, A. R. Bruun, and J. S. Persson, “Managing expectations towards AI tools for software development: a multiple-case study,” *Information Systems & e-Business Management*, pp. 1–33, 2025.

[28] U. Karlovs-Karlovskis, “Generative Artificial Intelligence Use in Optimising Software Engineering Process: A Systematic Literature Review,” *Applied Computer Systems*, vol. 29, no. 1, pp. 68–77, 2024.

[29] D.-K. Kim and H. Ming, “Assessing output reliability and similarity of large language models in software development: A comparative case study approach,” *Information and Software Technology*, vol. 185, 2025.

[30] D.-K. Kim, “Improving Software Development Traceability with Structured Prompting,” *Journal of Computer Information Systems*, pp. 1–21, 2025.

[31] J. Li, G. Li, Y. Li, and Z. Jin, “Structured Chain-of-Thought Prompting for Code Generation,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 2, pp. 1–23, 2025.

[32] V. S. Pendyala and N. B. Thakur, “Performance and interpretability analysis of code generation large language models,” *Neurocomputing*, vol. 656, 2025.

[33] K. Qiu, N. Puccinelli, M. Ciniselli, and L. Di Grazia, “From Today’s Code to Tomorrow’s Symphony: The AI Transformation of Developer’s Routine by 2030,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–17, 2025.

[34] T. Ramos, A. Dean, and D. McGregor, “AI-Augmented Software Engineering: Revolutionizing or Challenging Software Quality and Testing?” *Journal of Software: Evolution & Process*, vol. 37, no. 2, pp. 1–7, 2025.

[35] V. Terragni, A. Vella, P. Roop, and K. Blincoe, “The Future of AI-Driven Software Engineering,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–20, 2025.

[36] A. Tkachuk, “Determining the Capabilities of Generative Artificial Intelligence Tools to Increase the Efficiency of Refactoring Process,” *Technology Audit & Production Reserves*, vol. 3, no. 2(83), pp. 6–11, 2025.

[37] Q. Wang, J. Wang, M. Li, Y. Wang, and Z. Liu, “A Roadmap for Software Testing in Open-Collaborative and AI-Powered Era,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–17, 2025.

[38] M. Weyssow, X. Zhou, K. Kim, D. Lo, and H. Sahraoui, “Exploring Parameter-Efficient Fine-Tuning Techniques for Code Generation with Large Language Models,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 7, pp. 1–25, 2025.

[39] S. H. Yoo and H. J. Kim, “Security Analysis of Automated Code Generation: Structural Vulnerabilities in AI-Generated Code,” *Tehnički Glasnik*, vol. 19, no. 4, pp. 560–574, 2025.

[40] Thoughtworks, “Spec-driven development,” Technology Radar, Vol. 33, November 2025. [Online]. Available: https://www.thoughtworks.com/en-us/radar/techniques/spec-driven-development

[41] S. E. Farrag, “The Productivity-Reliability Paradox: Specification-Driven Governance for AI-Augmented Software Development,” arXiv:2605.01160, preprint, not yet peer-reviewed, 2026. [Online]. Available: https://arxiv.org/abs/2605.01160

[42] Amazon Web Services, “AI-Driven Development Lifecycle (AI-DLC),” Vendor methodology announced at AWS DevSphere 2025. Reported productivity figures are vendor-reported and not independently verified, 2025. [Online]. Available: https://aws.amazon.com/blogs/devops/ai-driven-development-life-cycle/

[43] J. Brooke, “SUS: A ‘quick and dirty’ usability scale,” in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. Taylor and Francis, 1996, pp. 189–194.

[44] A. I. Martins, A. F. Rosa, A. Queirós, A. Silva, and N. P. Rocha, “European Portuguese validation of the System Usability Scale (SUS),” *Procedia Computer Science*, vol. 67, pp. 293–300, 2015. [Online]. Available: https://doi.org/10.1016/j.procs.2015.09.273

[45] A. Bangor, P. Kortum, and J. Miller, “Determining what individual SUS scores mean: Adding an adjective rating scale,” *Journal of Usability Studies*, vol. 4, no. 3, pp. 114–123, 2009.

[46] J. Sauro and J. R. Lewis, *Quantifying the User Experience: Practical Statistics for User Research*, 2nd ed. Morgan Kaufmann, 2016.

[47] J. R. Lewis and J. Sauro, “The factor structure of the System Usability Scale,” in *Human Centered Design, HCII 2009, Lecture Notes in Computer Science*, vol. 5619, 2009, pp. 94–103.

[48] S. G. Hart and L. E. Staveland, “Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research,” in *Human Mental Workload*, P. A. Hancock and N. Meshkati, Eds. North-Holland, 1988, pp. 139–183.

[49] S. G. Hart, “NASA-Task Load Index (NASA-TLX); 20 years later,” in *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 50, 2006, pp. 904–908.

A

Systematic Literature Review Corpus

List the final set of studies retained by the review: identifier, full reference, venue, year, and which research questions each study contributes to. This is the auditability requirement of Kitchenham's guidelines - a reader must be able to reconstruct the review from this appendix together with the protocol reported in Chapter 4.

Include the full search string and the date on which the search was executed, if these are not already stated in the review protocol.

B

Evaluation Instruments

Reproduce the SUS questionnaire exactly as administered, including the response scale and any wording adapted to name the system under evaluation.

Reproduce the NASA-TLX instrument as administered, including the subscale definitions shown to participants and, if the weighted procedure was used, the pairwise comparison sheet.

Include the semi-structured interview guides for both participant groups (practitioners, and Agile and Scrum experts), and the informed consent form.

Include the per-participant raw scores in anonymised form, so the aggregate figures reported in Section 9.4 can be independently recomputed.